数据库设计与开发

Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

关系数据库标准语言SQL

Relational Database Language SQL

计算机学院　单纯
sherryshan@bit.edu.cn
2024年4月

内容概览

- 6.1 SQL简介
- 6.2 SQL的系统结构
- 6.3 学生-课程数据库
- 6.4 SQL的数据定义
- 6.5 SQL的数据操纵
- 6.6 SQL中的视图
- 6.7 SQL的数据控制
- 6.8 嵌入式SQL
- 6.9 小结

6.1 SQL简介（1）

- SQL（Structured Query Language）结构化查询语言，是关系数据库的标准语言
- SQL是一个通用的、功能极强的关系数据库语言

6.1 SQL简介（2）

- 6.1.1 SQL 的产生与发展
- 6.1.2 SQL的特点

6.1.1 SQL 的产生与发展（1）

- SQL语言的前身是SQUARE（Specifying Queries As Relational Expression）语言，是1974年由Boyce和Chamberlin提出的，并在IBM公司的关系数据库系统System R上实现，后改名为SEQUEL（Structured English QUEry Language）语言，SEUQUEL简称SQL

6.1.1 SQL 的产生与发展（2）

- 当我们去别的国家旅行时，需要了解其语言才能更加方便。举例来说，如果说服务员只能使用其本国语言，那我们用母语点菜可能就会有麻烦。如果把数据库看作一个要从中进行信息搜索的外国，那么SQL就是我们向数据库表达需求的语言，我们可以利用SQL进行查询，从数据库里获得特定的信息

6.1.2 SQL的特点

- 1. 综合统一
- 2. 高度非过程化
- 3. 面向集合的操作方式
- 4. 以同一种语法结构提供多种使用方式
- 5. 语言简洁，易学易用

1. 综合统一（1）

- SQL语言集数据定义语言DDL、数据操纵语言DML、数据控制语言DCL的功能于一体
  - 非关系模型（层次模型、网状模型）的数据语言一般分为
    - 模式数据定义语言（模式DDL）
    - 外模式数据定义语言（外模式DDL或子模式DDL）
    - 与数据存储有关的描述语言（DSDL）
    - 数据操纵语言（DML）

1. 综合统一（2）

- SQL语言集数据定义语言DDL、数据操纵语言DML、数据控制语言DCL的功能于一体（续）
  - 语言风格统一，可以独立完成数据库生命周期中的全部活动，包括
    - 定义关系模式，插入数据，建立数据库
    - 对数据库中的数据进行查询和更新
    - 数据库重构和维护
    - 数据库安全性、完整性控制等
  - 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据库的运行

1. 综合统一（3）

- 数据操作符统一
  - 查找、插入、删除、更新等每一种操作只需一种操作符，克服了非关系系统由于信息表示方式的多样性带来的操作复杂性

2. 高度非过程化

- 非关系数据模型的数据操纵语言“面向过程”，必须指定存取路径
- SQL只要提出“做什么”，无须了解存取路径
- 存取路径的选择以及SQL语句的操作过程由系统自动完成，大大减轻了用户负担，而且有利于提高数据独立性

3. 面向集合的操作方式

- 非关系数据模型采用面向记录的操作方式，操作对象是一条记录
- SQL采用集合操作方式
  - 操作对象、查找结果可以是元组的集合
  - 一次插入、删除、更新操作的对象可以是元组的集合

4. 以同一种语法结构提供多种使用方式

- SQL是独立的语言
  - 能够独立地用于联机交互的使用方式
- SQL又是嵌入式语言
  - SQL能够嵌入到高级语言（例如C，C++，Java）程序中，供程序员设计程序时使用
- 两种不同使用方式下，SQL语言的语法结构基本一致

5. 语言简洁，易学易用

- SQL功能极强，完成核心功能只用了9个动词

SQL 功 能	动 词
数据查询	SELECT
数据定义	CREATE，DROP，ALTER
数据操纵	INSERT，UPDATE DELETE
数据控制	GRANT，REVOKE

6.2 SQL的系统结构（1）

- SQL语言支持数据库的三级模式结构。在SQL中，关系模式称为“基本表（Base Table）”，基本表的集合形成数据库模式，对应三级模式结构的全局逻辑级。基本表在物理上与存储文件相对应，所有存储文件的集合为物理数据库。局部逻辑级由“视图（View）”组成，视图是定义在一个或多个基本表上的表，它本身不独立存储在数据库中，称为“虚表”

6.2 SQL的系统结构（2）

- 基本表
  - 本身独立存在的表
  - SQL中一个关系就对应一个基本表
  - 基本表对应存储文件
  - 一个表可以带若干索引
- 存储文件
  - 逻辑结构组成了关系数据库的内模式
  - 物理结构是任意的，对用户透明

6.2 SQL的系统结构（3）

- 视图
  - 从一个或几个基本表导出的表
  - 数据库中只存放视图的定义而不存放视图对应的数据
  - 视图是一个虚表
  - 用户可以在视图上再定义视图

6.3 学生-课程数据库

- 学生-课程模式 S-T
  - 学生表
    - Student（Sno，Sname，Ssex，Sage，Sdept）
  - 课程表
    - Course（Cno，Cname，Cpno，Ccredit）
  - 学生选课表
    - SC（Sno，Cno，Grade）

Student表

学 号 Sno	姓 名 Sname	性 别 Ssex	年 龄 Sage	所在系 Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

Course表

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

SC表

学 号 Sno	课程号 Cno	成绩 Grade
95001	1	92
95001	2	85
95001	3	88
95002	2	90
95002	3	80

6.4 SQL的数据定义（1）

- SQL的数据定义功能
  - 定义模式
    - 创建模式
    - 删除模式
  - 定义表（模式）
    - 创建表
    - 删除表
    - 修改表定义

6.4 SQL的数据定义（2）

- SQL的数据定义功能（续）
  - 定义视图（外模式）
    - 创建视图
    - 删除视图
    - 间接修改视图定义：删除+创建
  - 定义索引（内模式）
    - 创建索引
    - 删除索引
    - 间接修改索引定义：删除+创建

6.4 SQL的数据定义（3）

- SQL的数据定义语句

操作对象	操作方式		
	创建	删除	修改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	

6.4 SQL的数据定义（4）

- 6.4.1 SQL模式的定义和删除
- 6.4.2 基本表的定义、修改和删除
- 6.4.3 索引的建立和删除

6.4.1 SQL模式的定义和删除

- 一、定义模式
- 二、删除模式

一、定义模式（1）

[例1]定义一个学生-课程模式S-T

CREATE SCHEMA “S-T” AUTHORIZATION SHERRY；

为用户SHERRY定义了一个模式S-T

[例2]CREATE SCHEMA AUTHORIZATION SHERRY；

<模式名>隐含为用户名SHERRY

如果没有指定<模式名>，那么<模式名>隐含为<用户名>

一、定义模式（2）

- 定义模式实际上定义了一个命名空间
- 在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等
- 在CREATE SCHEMA中可以接受CREATE TABLE，CREATE VIEW和GRANT子句

CREATE SCHEMA <模式名>
AUTHORIZATION <用户名>[<表定义子句>|<视图定义子句>|<授权定义子句>]

一、定义模式（3）

[例3]

```
CREATE SCHEMA TEST AUTHORIZATION SHERRY
CREATE TABLE TAB1（COL1 SMALLINT，
                   COL2 INT，
                   COL3 CHAR（20），
                   COL4 NUMERIC（10，3），
                   COL5 DECIMAL（5，2）
                  ）；
```

为用户SHERRY创建了一个模式TEST，并在其中定义了一个表TAB1

二、删除模式（1）

- DROP SCHEMA <模式名> <CASCADE|RESTRICT>
  - CASCADE（级联）
    - 删除模式的同时把该模式中所有的数据库对象全部删除
  - RESTRICT（限制）
    - 如果该模式中定义了下属的数据库对象（如表、视图等），则拒绝该删除语句的执行
    - 当该模式中没有任何下属的对象时 才能执行

二、删除模式（2）

[例4] DROP SCHEMA SHERRY CASCADE；

- 删除模式SHERRY
- 同时该模式中定义的表TAB1也被删除

6.4.2 基本表的定义、修改和删除

- 一、定义基本表
- 二、数据类型
- 三、模式与表
- 四、修改基本表
- 五、删除基本表

一、定义基本表

- 语句格式

  CREATE TABLE <表名>

  （<列名> <数据类型>[<列级完整性约束条件>]

  [，<列名> <数据类型>[<列级完整性约束条件>]] …

  [，<表级完整性约束条件>] ）；

  - <表名>：所要定义的基本表的名字
  - <列名>：组成该表的各个属性（列）
  - <列级完整性约束条件>：涉及相应属性列的完整性约束条件
  - <表级完整性约束条件>：涉及一个或多个属性列的完整性约束条件

学生表Student

[例5] 建立“学生”表Student，学号是主键，姓名取值唯一

```
CREATE TABLE Student
  (Sno   CHAR(5)PRIMARY KEY, /* 列级完整性约束条件*/
   Sname  CHAR(20) UNIQUE,    /* Sname取唯一值*/
   Ssex   CHAR(2),
   Sage   SMALLINT,
   Sdept  CHAR(20));
```

课程表Course

[例6] 建立一个“课程”表Course

```
CREATE TABLE  Course
 (Cno  CHAR(4)PRIMARY KEY,
  Cname  CHAR(40),
  Cpno  CHAR(4),
  Ccredit  SMALLINT,
  FOREIGN KEY (Cpno)REFERENCES    Course(Cno));
```

学生选课表SC

[例7] 建立一个“学生选课”表SC

```
CREATE TABLE  SC
 (Sno  CHAR(5),
  Cno  CHAR(4),
  Grade   SMALLINT, PRIMARY KEY (Sno,Cno) ,
        /* 主键由两个属性构成，必须作为表级完整性进行定义*/
  FOREIGN KEY (Sno)REFERENCES Student(Sno),
        /* 表级完整性约束条件，Sno是外键，被参照表是Student */
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
        /* 表级完整性约束条件， Cno是外键，被参照表是Course*/);
```

二、数据类型（1）

- SQL中域的概念用数据类型来实现
- 定义表的属性时 需要指明其数据类型及长度
- 选用哪种数据类型
  - 取值范围
  - 要做哪些运算

二、数据类型（2）

数据类型	含义
CHAR(n)	长度为n的定长字符串
VARCHAR(n)	最大长度为n的变长字符串
INT	长整数（也可以写作INTEGER）
SMALLINT	短整数
NUMERIC(p，d)	定点数，由p位数字（不包括符号、小数点）组成，小数后面有d位数字
REAL	取决于机器精度的浮点数
Double Precision	取决于机器精度的双精度浮点数
FLOAT(n)	浮点数，精度至少为n位数字
DATE	日期，包含年、月、日，格式为YYYY-MM-DD
TIME	时间，包含一日的时、分、秒，格式为HH:MM:SS

二、数据类型（3）

- 不同的数据库系统支持的数据类型不完全相同
  - IBM DB2 SQL支持的数据类型
    - SMALLINT 半字长二进制整数
    - INTEGER或INT 全字长二进制整数
    - DECIMAL（p[，q]）或DEC （p[，q]） 压缩十进制数，共p位，其中小数点后有q位。0≤q≤p≤15，q=0时可以省略不写
    - FLOAT 双字长浮点数
    - CHARTER（n）或CHAR（n） 长度为n的定长字符串

二、数据类型（4）

- 不同的数据库系统支持的数据类型不完全相同（续）
  - IBM DB2 SQL支持的数据类型（续）
    - VARCHAR（n） 最大长度为n的变长字符串
    - GRAPHIC（n） 长度为n的定长图形字符串
    - VARGRAPHIC（n） 最大长度为n的变长图形字符串
    - DATE 日期型，格式为YYYY-MM-DD
    - TIME 时间型，格式为HH.MM.SS
    - TIMESTAMP 日期加时间

二、数据类型（5）

- 不同的数据库系统支持的数据类型不完全相同（续）
  - ORACLE SQL支持的数据类型
    - VARCHAR2（n）
    - CHAR（n）
    - NUMBER[（p[，q]）]
    - DATE
    - LONG
    - RAW或LONGRAW

三、模式与表（1）

- 每一个基本表都属于某一个模式
- 一个模式包含多个基本表
- 定义基本表所属模式
  - 方法一：在表名中明显地给出模式名
    Create table “S-T”.Student（.....）； /*模式名为S-T*/
    Create table “S-T”.Cource（.....）；
    Create table “S-T”.SC（.....）；
  - 方法二：在创建模式语句中同时创建表
  - 方法三：设置所属的模式

三、模式与表（2）

- 创建基本表（其他数据库对象也一样）时，若没有指定模式，系统根据搜索路径来确定该对象所属的模式
- RDBMS会使用模式列表中第一个存在的模式作为数据库对象的模式名
- 若搜索路径中的模式名都不存在，系统将给出错误

三、模式与表（3）

- 显示当前的搜索路径
  SHOW search_path；
- 搜索路径的当前默认值是
  $user， PUBLIC

三、模式与表（4）

- DBA用户可以设置搜索路径，然后定义基本表
  SET search_path TO “S-T”，PUBLIC；
  Create table Student（.....）；
  - 结果建立了S-T.Student基本表
  - RDBMS发现搜索路径中第一个模式名S-T存在，就把该模式作为基本表Student所属的模式

四、修改基本表（1）

- 语句格式
  ALTER TABLE <表名>
  [ADD <新列名> <数据类型> [<完整性约束 >]]
  [DROP <列名> [CASCADE | RESTRICT]]
  [ALTER <列名> <数据类型>]；
  - <表名>：要修改的基本表
  - ADD子句：增加新列和新的完整性约束条件
  - DROP子句：删除指定的列名
  - ALTER子句：用于修改列名和数据类型

四、修改基本表（2）

[例8]向Student表增加“入学时间”列，其数据类型为日期型
ALTER TABLE Student ADD S_entrance DATE；
- 不论基本表中原来是否已有数据，新增加的列一律为空值

- 注意：只能间接删除属性列
  - 把表中要保留的列及其内容复制到一个新表中
  - 删除原表
  - 再将新表重命名为原表名

四、修改基本表（3）

[例9] 将年龄的数据类型由字符型（假设原来的数据类型是字符型）改为整数

ALTER TABLE Student ALTER COLUMN Sage INT；

- 注：修改原有的列定义有可能会破坏已有数据

[例10] 增加课程名称必须取唯一值的约束条件

ALTER TABLE Course ADD UNIQUE（Cname）；

五、删除基本表（1）

- 语句格式

DROP TABLE <表名>［RESTRICT| CASCADE］；

- RESTRICT：删除表是有限制的
  - 欲删除的基本表不能被其他表的约束所引用
  - 如果存在依赖该表的对象，则此表不能被删除
- CASCADE：删除该表没有限制
  - 在删除基本表的同时，相关的依赖对象一起删除

五、删除基本表（2）

[例11] 删除Student表

DROP TABLE Student CASCADE；

- 基本表定义被删除，数据被删除
- 表上建立的索引、视图、触发器等一般也将被删除

五、删除基本表（3）

[例12] 若表上建有视图，选择RESTRICT时表不能删除

```
CREATE VIEW IS_Student
AS
  SELECT Sno，Sname，Sage
   FROM Student
   WHERE Sdept='IS'；

  DROP TABLE Student RESTRICT；
    ERROR: cannot drop table Student because other
           objects depend on it
```

五、删除基本表（4）

[例12] 如果选择CASCADE时可以删除表，视图也自动被删除

DROP TABLE Student CASCADE；

- NOTICE: drop cascades to view IS_Student

SELECT * FROM IS_Student；

- ERROR: relation "IS_Student " does not exist

6.4.3 索引的建立和删除（1）

- 建立索引的目的：加快查询速度
- 谁可以建立索引
  - DBA或表的属主（即建立表的人）
  - DBMS一般会自动建立以下列上的索引
    - PRIMARY KEY
    - UNIQUE

6.4.3 索引的建立和删除（2）

- 谁维护索引
  - DBMS自动完成
- 谁使用索引
  - DBMS自动选择是否使用索引以及使用哪些索引

6.4.3 索引的建立和删除（3）

- RDBMS中索引一般采用B+树、HASH索引来实现
  - B+树索引具有动态平衡的优点
  - HASH索引具有查找速度快的特点
- 采用B+树，还是HASH索引则由具体的RDBMS来决定

6.4.3 索引的建立和删除（4）

- 索引是关系数据库的内部实现技术，属于内模式的范畴
- CREATE INDEX语句定义索引时，可以定义索引是唯一索引、非唯一索引或聚簇索引

6.4.3 索引的建立和删除（5）

- 一、建立索引
- 二、删除索引

一、建立索引 （1）

- 语句格式
  - CREATE [UNIQUE] [CLUSTER] INDEX <索引名>　ON <表名>（<列名>[<次序>][,<列名>[<次序>]]…）；
    - 用<表名>指定要建索引的基本表名字
    - 索引可以建立在该表的一列或多列上，各列名之间用逗号分隔
    - 用<次序>指定索引值的排列次序，升序：ASC，降序：DESC，缺省值：ASC
    - UNIQUE表明此索引的每一个索引值只对应唯一的数据记录
    - CLUSTER表示要建立的索引是聚簇索引

一、建立索引 （2）

- 唯一值索引
  - 对于已含重复值的属性列不能建UNIQUE索引
  - 对某个列建立UNIQUE索引后，插入新记录时DBMS会自动检查新记录在该列上是否取了重复值。这相当于增加了一个UNIQUE约束
- 聚簇索引
  - 建立聚簇索引后，基表中数据也需要按指定的聚簇属性值的升序或降序存放。也即聚簇索引的索引项顺序与表中记录的物理顺序一致

一、建立索引（3）

[例13] CREATE CLUSTER INDEX Stusname ON Student（Sname）；

- 在Student表的Sname（姓名）列上建立一个聚簇索引

- 在最经常查询的列上建立聚簇索引以提高查询效率
- 一个基本表上最多只能建立一个聚簇索引
- 经常更新的列不宜建立聚簇索引

一、建立索引（4）

[例14] 为学生-课程数据库中的Student，Course，SC三个表建立索引

CREATE UNIQUE INDEX Stusno ON Student（Sno）；
CREATE UNIQUE INDEX Coucno ON Course（Cno）；
CREATE UNIQUE INDEX SCno ON SC（Sno ASC，Cno DESC）；

Student表按学号升序建唯一索引
Course表按课程号升序建唯一索引
SC表按学号升序和课程号降序建唯一索引

二、删除索引

- 语句格式
  - DROP INDEX <索引名>；
    - 删除索引时，系统会从数据字典中删去有关该索引的描述。

[例15] 删除Student表的Stusname索引
DROP INDEX Stusname；

6.5 SQL的数据操纵

- 6.5.1 数据查询
- 6.5.2 数据更新

6.5.1 数据查询

- 一、概述
- 二、单表查询
- 三、连接查询
- 四、嵌套查询
- 五、集合查询
- 六、SELECT语句的一般格式

一、概述（1）

- 语句格式

SELECT [ALL|DISTINCT] <目标列表达式>
[，<目标列表达式>] …
FROM <表名或视图名>[， <表名或视图名>] …
[WHERE <条件表达式>]
[GROUP BY <列名1> [HAVING <条件表达式>]]
[ORDER BY <列名2> [ASC|DESC]]；

一、概述（2）

- 语句格式（续）
  - SELECT子句：指定要显示的属性列
  - FROM子句：指定查询对象（基本表或视图）
  - WHERE子句：指定查询条件
  - GROUP BY子句：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用集函数
  - HAVING短语：筛选出只有满足指定条件的组
  - ORDER BY子句：对查询结果表按指定列值的升序或降序排序

示例数据库（1）

- 学生-课程数据库
  - 学生表
    Student（Sno，Sname，Ssex，Sage，Sdept）
  - 课程表
    Course（Cno，Cname，Cpno，Ccredit）
  - 学生选课表
    SC（Sno，Cno，Grade）

示例数据库（2）

Student

学 号 Sno	姓 名 Sname	性 别 Ssex	年 龄 Sage	所 在 系 Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

(a)

Course

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL 语言	6	4

(b)

SC

学 号 Sno	课 程 号 Cno	成 绩 Grade
95001	1	92
95001	2	85
95001	3	88
95002	2	90
95002	3	80

(c)

二、单表查询

- 查询仅涉及一个表
  - 选择表中的若干列
  - 选择表中的若干元组
  - ORDER BY子句
  - 聚集函数
  - GROUP BY子句

选择表中的若干列

- 1. 查询指定列
- 2. 查询全部列
- 3. 查询经过计算的值

1. 查询指定列

[例1] 查询全体学生的学号与姓名

```
SELECT Sno,Sname
FROM Student;
```

[例2] 查询全体学生的姓名、学号、所在系

```
SELECT Sname,Sno,Sdept
FROM Student;
```

2. 查询全部列

- 选出所有属性列
  - 在SELECT关键字后面列出所有列名
  - 将<目标列表达式>指定为 *

[例3] 查询全体学生的详细记录

```
SELECT  Sno,Sname,Ssex,Sage,Sdept
FROM Student;
```

或

```
SELECT  *
FROM Student;
```

3. 查询经过计算的值 （1）

- SELECT子句的<目标列表达式>可以为
  - 算术表达式
  - 字符串常量
  - 函数
  - 列别名

3. 查询经过计算的值 （2）

[例4] 查全体学生的姓名及其出生年份

```
SELECT  Sname,1996-Sage    /*假定当年的年份为1996年*/
FROM Student;
```

输出结果

Sname	1996-Sage
李勇	1976
刘晨	1977
王敏	1978
张立	1977

3. 查询经过计算的值 （3）

[例5] 查询全体学生的姓名、出生年份和所在系，要求用小写字母表示所在系名

```
SELECT  Sname,'Year of Birth:',1996-Sage,ISLOWER(Sdept)
FROM Student;
```

输出结果

Sname	'Year of Birth:'	1996-Sage	ISLOWER（Sdept）
李勇	Year of Birth:	1976	cs
刘晨	Year of Birth:	1977	is
王敏	Year of Birth:	1978	ma
张立	Year of Birth:	1977	is

3. 查询经过计算的值 （4）

- [例5.1] 使用列别名改变查询结果的列标题

```
SELECT Sname NAME,'Year of Birth: ' BIRTH,1996-Sage BIRTHDAY,LOWER(Sdept)  DEPARTMENT
FROM Student;
```

输出结果

NAME	BIRTH	BIRTHDAY	DEPARTMENT
李勇	Year of Birth:	1976	cs
刘晨	Year of Birth:	1977	is
王敏	Year of Birth:	1978	ma
张立	Year of Birth:	1977	is

选择表中的若干元组

- 1. 消除取值重复的行
- 2. 查询满足条件的元组
  - 比较大小
  - 确定范围
  - 确定集合
  - 字符匹配
  - 涉及空值的查询
  - 多重条件查询

1. 消除取值重复的行（1）

- 如果没有指定DISTINCT关键词，则缺省为ALL

[例6] 查询选修了课程的学生学号

SELECT Sno FROM SC;

等价于

SELECT ALL Sno FROM SC;

执行上面的SELECT语句后，结果为

Sno
95001
95001
95001
95002
95002

1. 消除取值重复的行（2）

- 指定DISTINCT关键词，去掉表中重复的行

SELECT DISTINCT Sno FROM SC;

执行结果

Sno
95001
95002

1. 消除取值重复的行（3）

- 注意： DISTINCT短语的作用范围是所有目标列
  - 例：查询选修课程的各种成绩
    - 错误的写法

      SELECT DISTINCT Cno,DISTINCT Grade
      FROM SC;
    - 正确的写法

      SELECT DISTINCT Cno,Grade
      FROM SC;

2. 查询满足条件的元组（1）

- 常用的查询条件

查询条件	谓 词
比 较	=，>，<，>=，<=，!=，<>，!>，!<；NOT+上述比较运算符
确定范围	BETWEEN AND，NOT BETWEEN AND
确定集合	IN，NOT IN
字符匹配	LIKE，NOT LIKE
空 值	IS NULL，IS NOT NULL
多重条件（逻辑运算）	AND，OR，NOT

2. 查询满足条件的元组（2）

A. 比较大小
B. 确定范围
C. 确定集合
D. 字符匹配
E. 涉及空值的查询
F. 多重条件查询

A. 比较大小（1）

[例7] 查询计算机科学系全体学生的名单

SELECT Sname
FROM Student
WHERE Sdept='CS';

[例8] 查询所有年龄在20岁以下的学生姓名及其年龄

SELECT Sname,Sage
FROM Student
WHERE Sage < 20;

A. 比较大小（2）

[例9] 查询考试成绩有不及格的学生的学号

```
SELECT DISTINCT Sno
FROM  SC
WHERE Grade<60;
```

B. 确定范围（1）

- 谓词
  - BETWEEN … AND …
  - NOT BETWEEN … AND …

[例10] 查询年龄在20~23岁（包括20岁和23岁）之间的学生的姓名、系别和年龄

```
SELECT Sname,Sdept,Sage
FROM    Student
WHERE   Sage BETWEEN 20 AND 23;
```

B. 确定范围（2）

[例11] 查询年龄不在20~23岁之间的学生姓名、系别和年龄

```
SELECT Sname,Sdept,Sage
FROM    Student
WHERE Sage NOT BETWEEN 20 AND 23;
```

C. 确定集合（1）

- 谓词
  - IN <值表>
  - NOT IN <值表>

[例12] 查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别

```
SELECT Sname,Ssex
FROM  Student
WHERE Sdept IN ('IS','MA','CS');
```

C. 确定集合（2）

[例13] 查询既不是信息系、数学系，也不是计算机科学系的学生的姓名和性别

```
SELECT    Sname,Ssex
FROM Student
WHERE Sdept  NOT IN('IS','MA','CS');
```

D. 字符匹配（1）

- 谓词
  - [NOT] LIKE ‘<匹配串>’ [ESCAPE ‘ <换码字符>’]
    - <匹配串>：指定匹配模板
      - 匹配模板：固定字符串或含通配符的字符串
        - 当匹配模板为固定字符串时，可以用 = 运算符取代 LIKE 谓词，用 != 或 < >运算符取代 NOT LIKE 谓词
    - ESCAPE短语
      - 当用户要查询的字符串本身就含有 % 或 _ 时，要使用 ESCAPE '<换码字符>' 短语对通配符进行转义

D. 字符匹配（2）

- 谓词

  [NOT] LIKE ‘<匹配串>’ [ESCAPE ‘ <换码字符>’]

  （续）

  - 通配符
    - % （百分号）代表任意长度（长度可以为0）的字符
      - 例：a%b表示以a开头，以b结尾的任意长度的字符串。如acb，addgb，ab 等都满足该匹配串
    - _ （下横线)）代表任意单个字符
      - 例：a_b表示以a开头，以b结尾的长度为3的任意字符串。如acb，afb等都满足该匹配串

D. 字符匹配（3）

- D.1 匹配串为固定字符串
- D.2 匹配串为含通配符的字符串
- D.3 使用换码字符将通配符转义为普通字符

D.1 匹配串为固定字符串

- [例14] 查询学号为95001的学生的详细情况

```
SELECT *
FROM  Student
WHERE  Sno  LIKE '95001';
```

等价于

```
SELECT  *
FROM  Student
WHERE Sno = '95001';
```

D.2 匹配串为含通配符的字符串（1）

[例15] 查询所有姓刘学生的姓名、学号和性别

```
SELECT Sname,Sno,Ssex
FROM Student
WHERE  Sname LIKE '刘%';
```

[例16] 查询姓"欧阳"且全名为三个汉字的学生的姓名

```
SELECT Sname
FROM   Student
WHERE  Sname LIKE '欧阳__';
```

D.2 匹配串为含通配符的字符串（2）

[例17] 查询名字中第2个字为"阳"字的学生的姓名和学号

```
SELECT Sname,Sno
FROM Student
WHERE Sname LIKE '__阳%';
```

[例18] 查询所有不姓刘的学生姓名

```
SELECT Sname,Sno,Ssex
FROM Student
WHERE Sname NOT LIKE '刘%';
```

D.3 使用换码字符将通配符转义为普通字符（1）

[例19] 查询DB_Design课程的课程号和学分

```
SELECT Cno,Ccredit
FROM Course
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```

D.3 使用换码字符将通配符转义为普通字符（2）

[例20] 查询以"DB_"开头，且倒数第3个字符为 i的课程的详细情况

```
SELECT *
FROM Course
WHERE Cname LIKE 'DB\_%i_ _' ESCAPE '\';
```

ESCAPE '\' 表示 “ \ ” 为换码字符

E. 涉及空值的查询（1）

- 谓词
  IS NULL
  IS NOT NULL
- “IS” 不能用 “=” 代替

E. 涉及空值的查询（2）

[例21] 某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NULL;
/*分数Grade是空值*/
```

E. 涉及空值的查询（3）

[例22] 查所有有成绩的学生学号和课程号

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NOT NULL;
```

F. 多重条件查询（1）

- 逻辑运算符：AND和 OR来连接多个查询条件
  – AND的优先级高于OR
  – 可以用括号改变优先级
- 可用来实现多种其他谓词
  [NOT] IN
  [NOT] BETWEEN … AND …

F. 多重条件查询（2）

[例23] 查询计算机系年龄在20岁以下的学生姓名

```
SELECT Sname
FROM Student
WHERE Sdept='CS' AND Sage<20;
```

F. 多重条件查询（3）

- 改写[例12]

  [例12] 查询信息系（IS）、数学系（MA）和计算机科学系（CS）学生的姓名和性别

```
SELECT Sname,Ssex
FROM Student
WHERE Sdept IN ('IS','MA','CS');
```

可改写为

```
SELECT Sname,Ssex
FROM   Student
WHERE  Sdept='IS'OR Sdept='MA' OR
Sdept='CS';
```

ORDER BY子句（1）

- ORDER BY子句
  - 可以按一个或多个属性列排序
  - 升序：ASC；降序：DESC；缺省值为升序
- 当排序列含空值时
  - ASC：排序列为空值的元组最后显示
  - DESC：排序列为空值的元组最先显示

ORDER BY子句（2）

[例24] 查询选修了3号课程的学生的学号及其成绩，查询结果按分数降序排列

```
SELECT Sno,Grade
FROM  SC
WHERE  Cno='3'
ORDER BY Grade DESC;
```

ORDER BY子句（3）

[例25] 查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列

```
SELECT  *
FROM  Student
ORDER BY Sdept,Sage DESC;
```

聚集函数（1）

- 计数

  COUNT（ *）

  COUNT（[DISTINCT|ALL] <列名>）
- 计算总和

  SUM（[DISTINCT|ALL] <列名>）
- 计算平均值

  AVG（[DISTINCT|ALL] <列名>）
- 最大最小值

  MAX（[DISTINCT|ALL] <列名>）

  MIN（[DISTINCT|ALL] <列名>）

聚集函数（2）

- DISTINCT短语：在计算时要取消指定列中的重复值
- ALL短语：不取消重复值
- ALL为缺省值

聚集函数（3）

[例26] 查询学生总人数

```
SELECT COUNT(*)
FROM  Student;
```

[例27] 查询选修了课程的学生人数

```
SELECT COUNT(DISTINCT Sno)
FROM SC;
```

注：用DISTINCT以避免重复计算学生人数

[例28] 计算1号课程的学生平均成绩

```
SELECT AVG(Grade)
FROM SC
WHERE Cno='1';
```

聚集函数（4）

[例29] 查询选修1号课程的学生最高分数

```
SELECT MAX(Grade)
FROM SC
WHERE Cno='1';
```

[例30]查询学生95001选修课程的总学分数

```
SELECT  SUM(Ccredit)
FROM  SC,Course
WHERE Sno='95001'AND
SC.Cno=Course.Cno;
```

GROUP BY子句（1）

- GROUP BY子句分组
  - 细化聚集函数的作用对象
    - 未对查询结果分组，聚集函数将作用于整个查询结果
    - 对查询结果分组后，聚集函数将分别作用于每个组
    - 作用对象是查询的中间结果表
    - 按指定的一列或多列值分组，值相等的为一组

GROUP BY子句（2）

- 用GROUP BY子句分组

[例31] 求各个课程号及相应的选课人数

```
SELECT Cno,COUNT(Sno)
FROM    SC
GROUP BY Cno;
```

GROUP BY子句（3）

- 用GROUP BY子句分组（续）
  - GROUP BY子句的作用对象是查询的中间结果表
  - 分组方法：按指定的一列或多列值分组，值相等的为一组
  - 使用GROUP BY子句后，SELECT子句的列名列表中只能出现分组属性和聚集函数

GROUP BY子句（4）

- 使用HAVING短语筛选最终输出结果

[例32] 查询选修了3门以上课程的学生学号

```
SELECT Sno
FROM  SC
GROUP BY Sno
HAVING  COUNT(*) >3;
```

GROUP BY子句（5）

- 使用HAVING短语筛选最终输出结果（续）

  [例32.1] 查询有3门以上课程是90分以上的学生的学号及（90分以上的）课程数

```
SELECT  Sno,COUNT(*)
FROM   SC
WHERE Grade>=90
GROUP BY Sno
HAVING COUNT(*)>=3;
```

GROUP BY子句（6）

- 使用HAVING短语筛选最终输出结果（续）
  - 只有满足HAVING短语指定条件的组才输出
  - HAVING短语与WHERE子句的区别：作用对象不同
    - WHERE子句作用于基表或视图，从中选择满足条件的元组
    - HAVING短语作用于组，从中选择满足条件的组

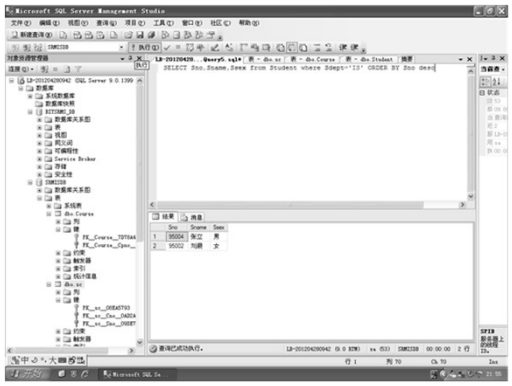
课堂练习1

- 查询信息系学生的学号、姓名、性别，要求结果按学号降序排列

参考答案

```
SELECT Sno,Sname,Ssex
FROM Student
WHERE Sdept= 'IS'
ORDER BY Sno DESC;
```

SQL Server运行结果



OpenGauss运行结果

```
postgres=# SELECT Sno,Sname,Ssex
postgres-# FROM Student
postgres-# WHERE Sdept= 'IS'
postgres-# ORDER BY Sno DESC;
  sno  |        sname        |        ssex
-------+---------------------+---------------------
 95004 | 张立                | 男
 95002 | 刘晨                | 女
(2 rows)
```

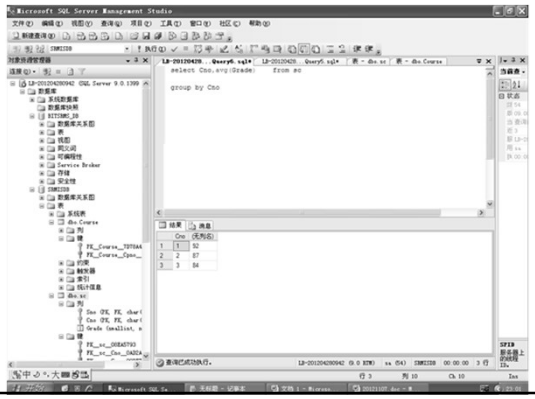
课堂练习2

- 查询每门课程的课程号及其平均成绩。要求结果按课程号分组显示

参考答案

```
SELECT Cno,AVG(Grade)
FROM SC
GROUP BY Cno;
```

SQL Server运行结果



OpenGauss运行结果

```
postgres=# SELECT Cno,AVG(Grade)
postgres-# FROM SC
postgres-# GROUP BY Cno;
 cno |          avg
-----+---------------------
   1 | 92.0000000000000000
   3 | 84.0000000000000000
   2 | 87.5000000000000000
(3 rows)
```

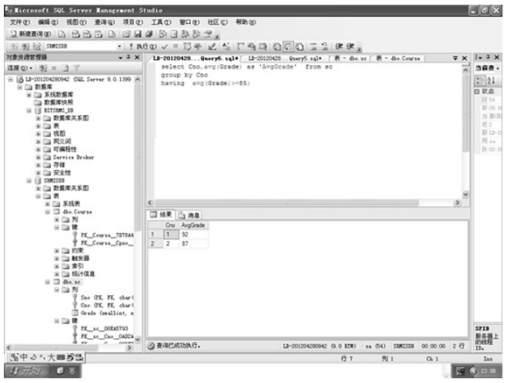
课堂练习3

- 查询每门课程的课程号及其平均成绩。要求结果按课程号分组显示，并且只显示平均成绩在85分（含）以上的组，平均成绩列名需使用‘AvgGrade’

参考答案

```
SELECT Cno,AVG(Grade) AS'AvgGrade'
FROM SC
GROUP  BY Cno
HAVING AVG(Grade)>=85;
```

SQL Server运行结果



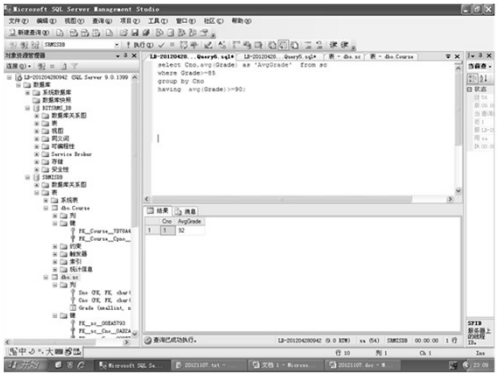
课堂练习4

- 查询得分在85分（含）以上课程的课程号及其平均成绩。要求结果按课程号分组显示，并且只显示平均成绩在90分（含）以上的组，平均成绩列名需使用‘AvgGrade’

参考答案

```
SELECT Cno,AVG(Grade) AS'AvgGrade'
FROM SC
WHERE Grade>=85
GROUP BY Cno
HAVING AVG(Grade) >=90;
```

SQL Server运行结果



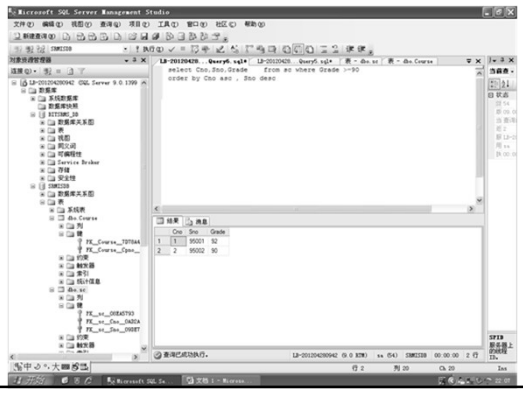
课堂练习5

- 查询成绩为优秀（>=90分）课程的课程号、学号及其成绩。要求结果按照课程号升序排列，同一课程号内按学号的降序排列

参考答案

```
SELECT Cno,Sno,Grade
FROM SC
WHERE Grade>=90
ORDER BY Cno ASC,Sno DESC;
```

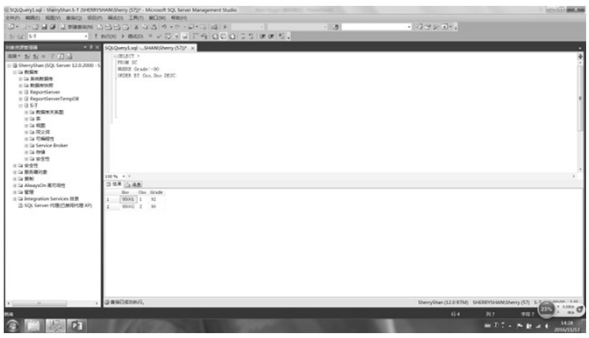
SQL Server运行结果



等价于

SELECT *
FROM SC
WHERE Grade>=90
ORDER BY Cno,Sno DESC;

SQL Server运行结果



OpenGauss运行结果

```
onclass=# select cno,sno,grade
onclass-# from sc
onclass-# where grade>=90
onclass-# order by cno asc,sno desc;
 cno |  sno  | grade
-----+-------+-------
   1 | 95001 |    92
   2 | 95001 |    90
(2 rows)
```

三、连接查询（1）

- 连接查询：同时涉及多个表的查询
- 连接条件或连接谓词：用来连接两个表的条件
- 一般格式

  [<表名1>.]<列名1> <比较运算符> [<表名2>.]<列名2>

  [<表名1>.]<列名1> BETWEEN [<表名2>.]<列名2> AND [<表名2>.]<列名3>

三、连接查询（2）

- 连接字段：连接谓词中的列名称
- 连接条件中的各连接字段类型必须是可比的，但名字不必是相同的

三、连接查询（3）

- 连接操作的执行过程
  - 嵌套循环法（NESTED-LOOP）
  - 排序合并法（SORT-MERGE）
  - 索引连接（INDEX-JOIN）

1. 嵌套循环法（NESTED-LOOP）

- 首先在表1中找到第一个元组，然后从头开始扫描表2，逐一查找满足连接条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组
- 表2全部查找完后，再找表1中第二个元组，然后再从头开始扫描表2，逐一查找满足连接条件的元组，找到后就将表1中的第二个元组与该元组拼接起来，形成结果表中一个元组
- 重复上述操作，直到表1中的全部元组都处理完毕

2. 排序合并法（SORT-MERGE）（1）

- 常用于=连接
  - 首先按连接属性对表1和表2排序
  - 对表1的第一个元组，从头开始扫描表2，顺序查找满足连接条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组。当遇到表2中第一条大于表1连接字段值的元组时，对表2的查询不再继续

2. 排序合并法（SORT-MERGE）（2）

- 常用于=连接（续）
  - 找到表1的第二条元组，然后从刚才的中断点处继续顺序扫描表2，查找满足连接条件的元组，找到后就将表1中的第二个元组与该元组拼接起来，形成结果表中一个元组。直接遇到表2中大于表1连接字段值的元组时，对表2的查询不再继续
  - 重复上述操作，直到表1或表2中的全部元组都处理完毕为止

3. 索引连接（INDEX-JOIN）

- 对表2按连接字段建立索引
- 对表1中的每个元组，依次根据其连接字段值查询表2的索引，从中找到满足条件的元组，找到后就将表1中的第一个元组与该元组拼接起来，形成结果表中一个元组

三、连接查询（4）

- 等值与非等值连接查询
- 自身连接
- 外连接
- 复合条件连接

1. 等值与非等值连接查询（1）

- 等值连接：连接运算符为=

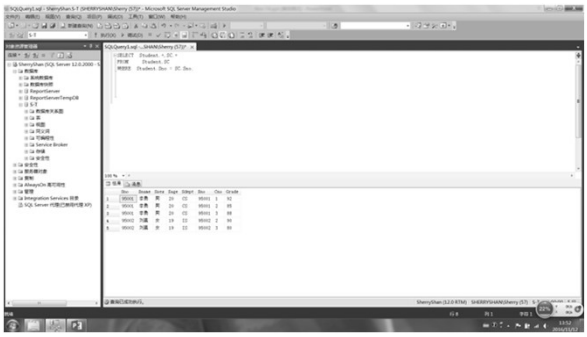
  [<表名1>.]<列名1> = [<表名2>.]<列名2>

  - 任何子句中引用表1和表2中同名属性时，都必须加表名前缀。引用唯一属性名时可以加也可以省略表名前缀

  [例33] 查询每个学生及其选修课程的情况

```
SELECT  Student.*,SC.*
FROM    Student,SC
WHERE   Student.Sno = SC.Sno;
```

运行结果



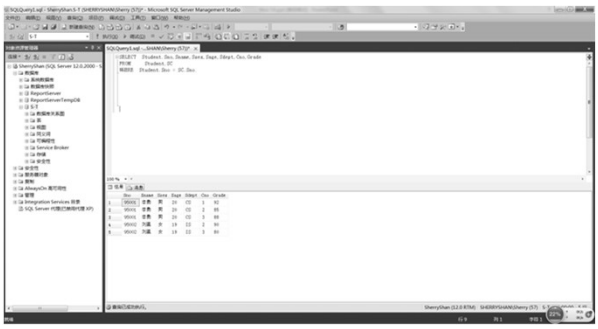
1. 等值与非等值连接查询（2）

- 自然连接
  - 等值连接的一种特殊情况，把目标列中重复的属性列去掉

  [例34] 对 [例33] 查询每个学生及其选修课程的情况用自然连接完成

```
SELECT
Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM    Student,SC
WHERE   Student.Sno = SC.Sno;
```

运行结果



2. 自身连接（1）

- 自身连接：一个表与其自己进行连接
  - 需要给表起别名以示区别
  - 由于所有属性名都是同名属性，因此必须使用别名前缀

  [例35] 查询每一门课的间接先修课（即先修课的先修课）

```
SELECT  FIRST.Cno,SECOND.Cpno
FROM  Course  FIRST,Course  SECOND
WHERE FIRST.Cpno = SECOND.Cno;
```

2. 自身连接（2）

- FIRST表（Course表）

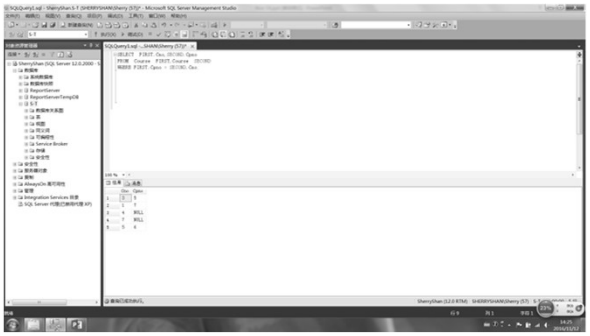
Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

2. 自身连接（3）

- SECOND表（Course表）

Cno	Cname	Cpno	Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

运行结果



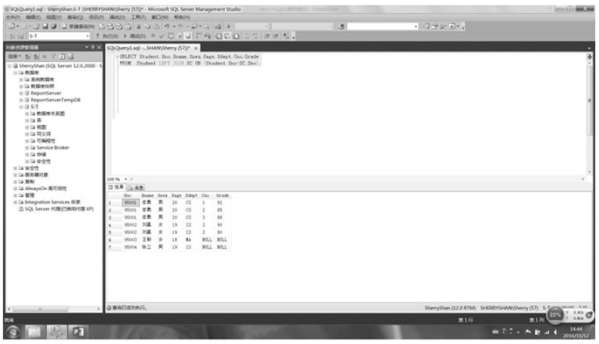
3. 外连接（1）

- 外连接与普通连接的区别
  - 普通连接操作只输出满足连接条件的元组
  - 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出

[例36]　改写[例33] 查询每个学生及其选修课程的情况，包括没有选修课程的学生

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM  Student LEFT OUTER JOIN SC ON (Student.Sno=SC.Sno);
```

运行结果



3. 外连接（2）

- 左外连接
  - 列出左边关系（如本例Student）中所有的元组
- 右外连接
  - 列出右边关系中所有的元组

4. 复合条件连接（1）

- 复合条件连接：WHERE子句中含多个连接条件

[例37] 查询选修2号课程且成绩在90分以上的所有学生

```
SELECT Student.Sno, Sname
FROM    Student, SC
WHERE Student.Sno = SC.Sno AND
SC.Cno='2'AND SC.Grade > 90;
```

运行结果

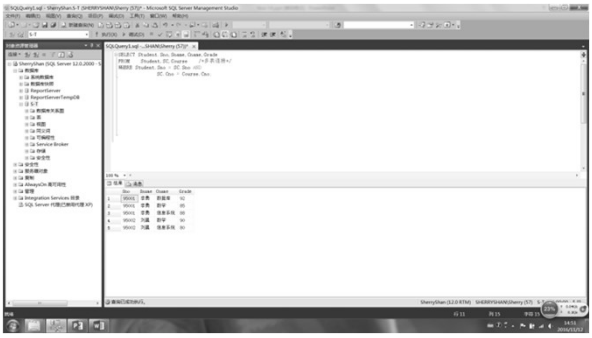
4. 复合条件连接（2）

- 连接操作除了可以是两表连接，一个表与其自身连接外，还可以是两个以上的表进行连接，后者通常称为多表连接

  [例38] 查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno,Sname,Cname,Grade
FROM    Student,SC,Course    /*多表连接*/
WHERE Student.Sno = SC.Sno AND
          SC.Cno = Course.Cno;
```

运行结果



四、嵌套查询

- 嵌套查询概述
- 嵌套查询分类
- 嵌套查询求解方法
- 引出子查询的谓词

嵌套查询概述（1）

- 一个SELECT-FROM-WHERE语句称为一个查询块
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询

嵌套查询概述（2）

```
SELECT Sname      /*外层查询/父查询*/
FROM Student
WHERE Sno IN
(SELECT Sno    /*内层查询/子查询*/
   FROM SC
   WHERE Cno='2');
```

运行结果

嵌套查询概述（3）

- 子查询的限制
  - 不能使用ORDER BY子句
- 层层嵌套方式反映了 SQL语言的结构化
- 有些嵌套查询可以用连接运算替代

嵌套查询分类

- 不相关子查询
  - 子查询的查询条件不依赖于父查询
- 相关子查询
  - 子查询的查询条件依赖于父查询

嵌套查询求解方法（1）

- 不相关子查询
  - 由里向外 逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件

嵌套查询求解方法（2）

- 相关子查询
  - 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若WHERE子句返回值为真，则取此元组放入结果表
  - 然后再取外层表的下一个元组
  - 重复这一过程，直至外层表全部检查完为止

引出子查询的谓词

- 1. 带有IN谓词的子查询
- 2. 带有比较运算符的子查询
- 3. 带有ANY或ALL谓词的子查询
- 4. 带有EXISTS谓词的子查询

1. 带有IN谓词的子查询（1）

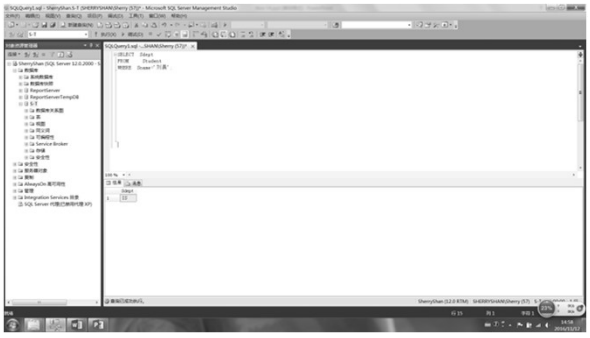
[例39] 查询与“刘晨”在同一个系学习的学生

– 此查询要求可以分步来完成

① 确定“刘晨”所在系名

```
SELECT  Sdept
FROM    Student
WHERE  Sname='刘晨';
```

运行结果

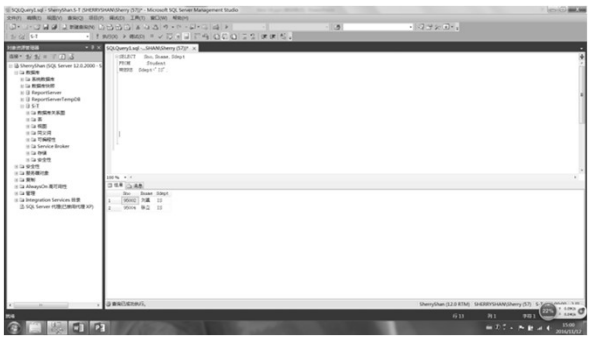


1. 带有IN谓词的子查询（2）

② 查找所有在IS系学习的学生

```
SELECT   Sno,Sname,Sdept
FROM     Student
WHERE  Sdept='IS';
```

运行结果



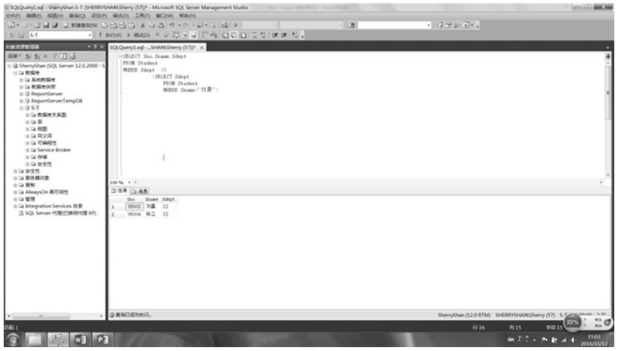
1. 带有IN谓词的子查询（3）

- 将第一步查询嵌入到第二步查询的条件中

```
SELECT Sno,Sname,Sdept
FROM Student
WHERE Sdept  IN
        (SELECT Sdept
           FROM Student
           WHERE Sname='刘晨');
```

- 此查询为不相关子查询。DBMS求解该查询时也是分步去做的

运行结果

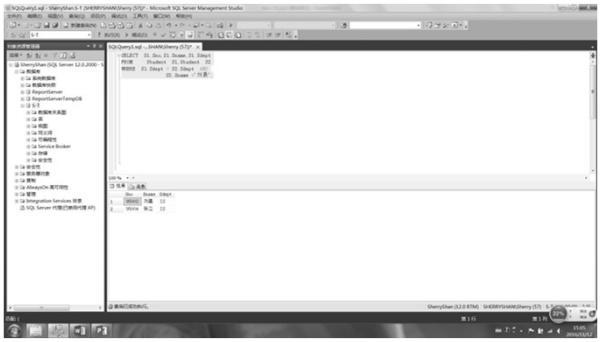


1. 带有IN谓词的子查询（4）

- 用自身连接完成[例39]查询要求

```
SELECT  S1.Sno,S1.Sname,S1.Sdept
FROM     Student  S1,Student  S2
WHERE  S1.Sdept = S2.Sdept  AND
            S2.Sname ='刘晨';
```

运行结果



1. 带有IN谓词的子查询（5）

[例40] 查询选修了课程名为“信息系统”的学生学号和姓名

```
SELECT Sno,Sname            /*③ 最后在Student关系中取出Sno和Sname*/
FROM    Student
WHERE Sno  IN
    (SELECT Sno             /*② 然后在SC关系中找出选修了3号课程的学生学号*/
      FROM    SC
      WHERE  Cno IN
      (SELECT Cno       /*① 首先在Course关系中找出“信息系统”的课程号，结果为3号*/
        FROM Course
        WHERE Cname='信息系统'
      )
    );
```

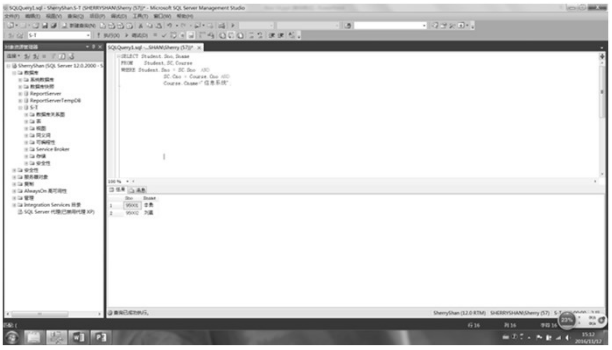
运行结果

1. 带有IN谓词的子查询（6）

- 用连接查询实现[例40]

```
SELECT Student.Sno,Sname
FROM    Student,SC,Course
WHERE Student.Sno = SC.Sno  AND
            SC.Cno = Course.Cno AND
            Course.Cname='信息系统';
```

运行结果



2. 带有比较运算符的子查询（1）

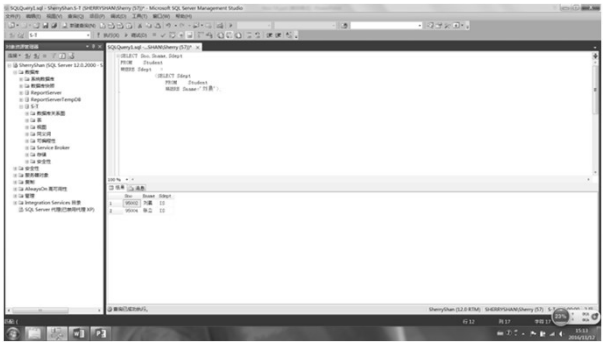
- 当能确切知道内层查询返回单值时，可用比较运算符（>，<，=，>=，<=，!=或< >）
- 与ANY或ALL谓词配合使用

2. 带有比较运算符的子查询（2）

- 例：假设一个学生只可能在一个系学习，并且必须属于一个系，则在[例39]可以用 = 代替IN

```
SELECT Sno,Sname,Sdept
FROM    Student
WHERE Sdept   =
        (SELECT Sdept
           FROM    Student
           WHERE Sname='刘晨');
```

运行结果



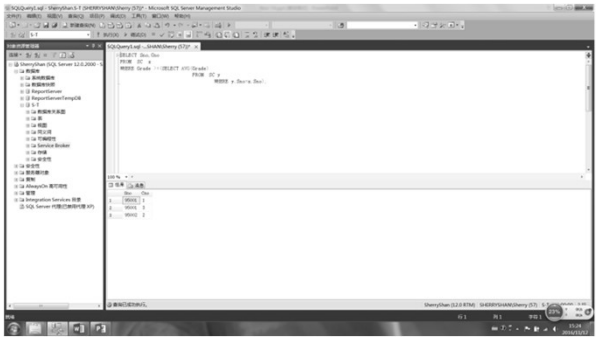
2. 带有比较运算符的子查询（3）

[例41]　找出每个学生超过他选修课程平均成绩的课程号

```
SELECT Sno,Cno
FROM  SC  x
WHERE Grade >=(SELECT AVG(Grade)
                FROM  SC y
                WHERE y.Sno=x.Sno);
```

/*x是表SC的别名，又称元组变量，可以用来表示SC的一个元组。内层查询是求一个学生所有选修课程平均成绩的，至于是哪个学生的平均成绩要看参数x.Sno的值，而该值是与父查询相关的，因此这类查询称为相关子查询。*/

运行结果



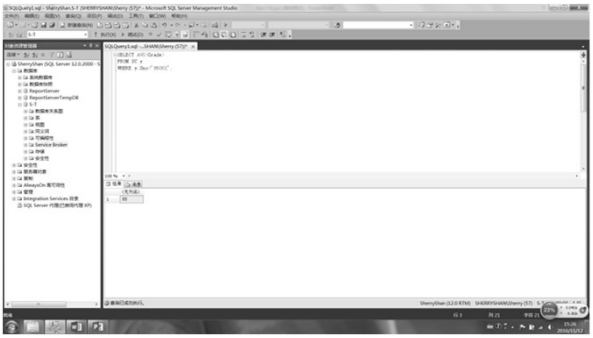
2. 带有比较运算符的子查询（4）

- 可能的执行过程

1. 从外层查询中取出SC的一个元组x，将元组x的Sno值（95001）传送给内层查询

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='95001';
```

运行结果



2. 带有比较运算符的子查询（5）

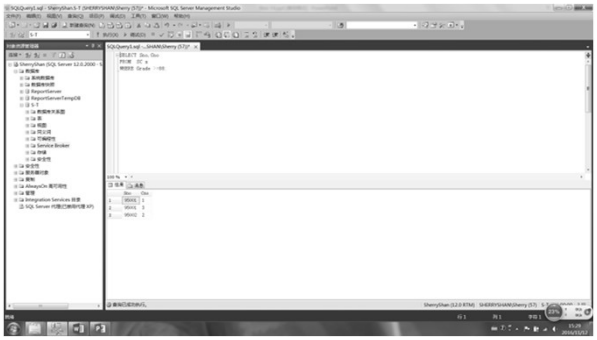
- 可能的执行过程（续）
  - 2. 执行内层查询，得到值88（近似值），用该值代替内层查询，得到外层查询

```
SELECT Sno,Cno
FROM  SC x
WHERE Grade >=88;
```

2. 带有比较运算符的子查询（6）

- 可能的执行过程 （续）
  - 3. 执行这个查询，得到
    （95001，1）
    （95001，3）
  - 4. 外层查询取出下一个元组重复做上述1至3步骤，直到外层的SC元组全部处理完毕。结果为
    （95001，1）
    （95001，3）
    （95002，2）

运行结果



3. 带有ANY或ALL谓词的子查询（1）

- 谓词语义
  ANY：任意一个值
  ALL：所有值

3. 带有ANY或ALL谓词的子查询（2）

- 需要配合使用比较运算符
  > ANY　大于子查询结果中的某个值
  > ALL　大于子查询结果中的所有值
  < ANY　小于子查询结果中的某个值
  < ALL　小于子查询结果中的所有值
  >= ANY 大于等于子查询结果中的某个值
  >= ALL 大于等于子查询结果中的所有值
  <= ANY 小于等于子查询结果中的某个值

3. 带有ANY或ALL谓词的子查询（3）

- 需要配合使用比较运算符（续）

  <= ALL 小于等于子查询结果中的所有值

  = ANY 等于子查询结果中的某个值

  =ALL 等于子查询结果中的所有值（通常没有实际意义）

  !=（或<>）ANY 不等于子查询结果中的某个值

  !=（或<>）ALL 不等于子查询结果中的任何一个值

3. 带有ANY或ALL谓词的子查询（4）

[例42] 查询其他系中比信息系某一学生年龄小的学生姓名和年龄

```
SELECT Sname,Sage
FROM   Student
WHERE Sage < ANY (SELECT  Sage
                  FROM   Student
                  WHERE Sdept='IS')
                AND Sdept <>'IS'; /*父查询块中的条件 */
```

运行结果

3. 带有ANY或ALL谓词的子查询（5）

- ANY和ALL谓词有时可以用集函数实现
  - ANY（或SOME），ALL谓词与聚集函数、IN谓词的等价转换关系

	=	<>或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX

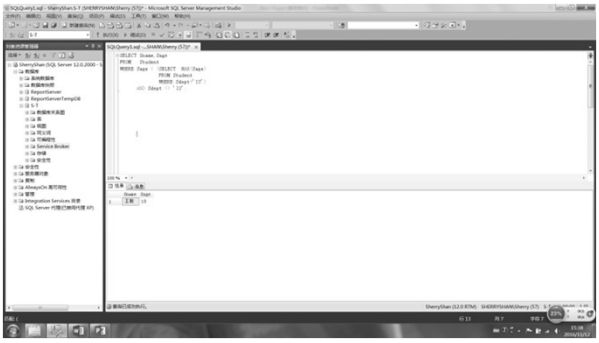
  - 用集函数实现子查询通常比直接用ANY或ALL查询效率要高，因为前者通常能够减少比较次数

3. 带有ANY或ALL谓词的子查询（6）

- 用聚集函数实现[例42]

```
SELECT Sname,Sage
FROM   Student
WHERE Sage < (SELECT  MAX(Sage)
              FROM Student
              WHERE Sdept='IS')
    AND Sdept <> 'IS';
```

运行结果



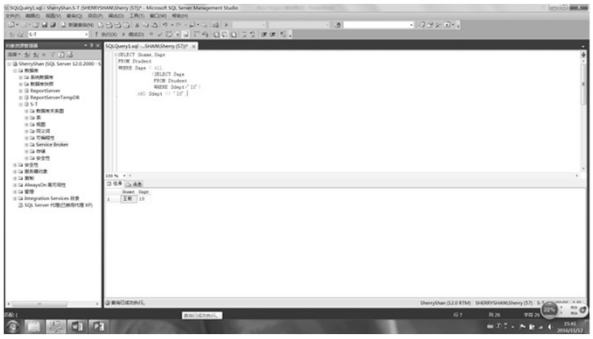
3. 带有ANY或ALL谓词的子查询（7）

[例43] 查询其他系中比信息系所有学生年龄都小的学生姓名及年龄

方法一：用ALL谓词

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ALL
        (SELECT Sage
         FROM Student
         WHERE Sdept='IS')
    AND Sdept <> 'IS';
```

运行结果

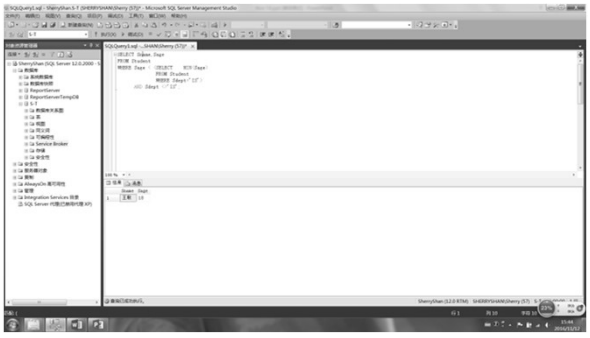


3. 带有ANY或ALL谓词的子查询（8）

方法二：用集函数

```
SELECT Sname,Sage
FROM Student
WHERE Sage < (SELECT    MIN(Sage)
              FROM Student
              WHERE Sdept='IS')
    AND Sdept <>'IS';
```

运行结果



4. 带有EXISTS谓词的子查询

- EXISTS谓词
- NOT EXISTS谓词
- 不同形式的查询间的替换
- 用EXISTS/NOT EXISTS实现全称量词
  - 难点
- 用EXISTS/NOT EXISTS实现逻辑蕴涵
  - 难点

EXISTS谓词（1）

- 存在量词
- 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”
  - 若内层查询结果非空，则外层的WHERE子句返回真值
  - 若内层查询结果为空，则外层的WHERE子句返回假值

EXISTS谓词（2）

- 由EXISTS引出的子查询，其目标列表达式通常都用*，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义

EXISTS谓词（3）

[例44] 查询所有选修了1号课程的学生姓名

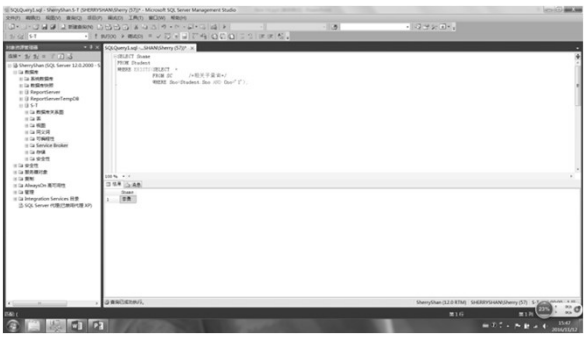
– 思路分析
  - 本查询涉及Student和SC关系
  - 在Student中依次取每个元组的Sno值，用此值去检查SC关系
  - 若SC中存在这样的元组，其Sno值等于此Student.Sno值，并且其Cno= ‘1’，则取此Student.Sname送入结果关系

EXISTS谓词（4）

- 用嵌套查询

```
SELECT Sname
FROM Student
WHERE EXISTS(SELECT  *
        FROM SC     /*相关子查询*/
        WHERE Sno=Student.Sno AND
Cno='1');
```

运行结果

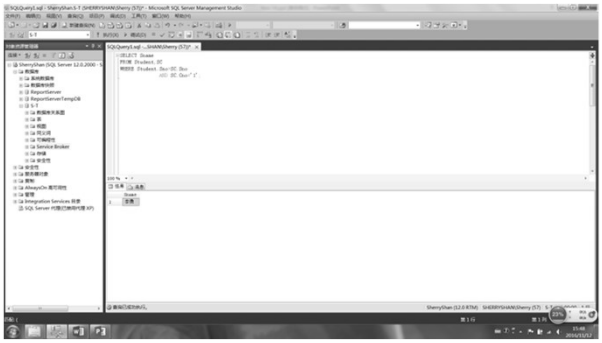


EXISTS谓词（5）

- 用连接运算

```
SELECT Sname
FROM Student,SC
WHERE Student.Sno=SC.Sno
        AND SC.Cno='1';
```

运行结果



NOT EXISTS谓词（1）

- 若内层查询结果非空，则外层的WHERE子句返回假值
- 若内层查询结果为空，则外层的WHERE子句返回真值

NOT EXISTS谓词（2）

[例45] 查询没有选修1号课程的学生姓名

```
SELECT  Sname
FROM Student
WHERE NOT EXISTS
        (SELECT *
         FROM SC
         WHERE Sno = Student.Sno AND
Cno='1');
```

- 此例用连接运算难于实现

运行结果

不同形式的查询间的替换（1）

- 一些带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
- 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换

不同形式的查询间的替换（2）

- 例：[例39]查询与“刘晨”在同一个系学习的学生（带有IN谓词的子查询）
  - 可以用带EXISTS谓词的子查询替换

```
SELECT Sno,Sname,Sdept
FROM Student S1
 WHERE EXISTS
        (SELECT *
         FROM Student S2
         WHERE S2.Sdept = S1.Sdept AND
S2.Sname ='刘晨');
```

运行结果

用EXISTS/NOT EXISTS实现全称量词（1）

- SQL语言中没有全称量词∀ （For all）
- 可以把带有全称量词的谓词转换为等价的带有存在量词的谓词

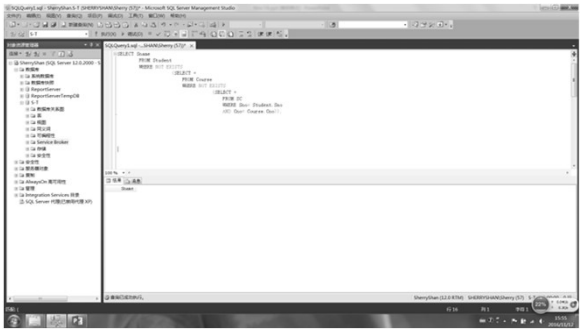
  （∀x）P ≡ ¬ （∃ x（¬ P））

用EXISTS/NOT EXISTS实现全称量词（2）

［例46］查询选修了全部课程的学生姓名

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
        (SELECT *
           FROM Course
           WHERE NOT EXISTS
                 (SELECT *
                    FROM SC
                    WHERE Sno= Student.Sno
                    AND Cno= Course.Cno));
```

运行结果



用EXISTS/NOT EXISTS实现逻辑蕴涵（1）

- SQL语言中没有蕴涵（Implication）逻辑运算
- 可以利用谓词演算将逻辑蕴涵谓词等价转换为

  $p \rightarrow q \equiv \neg p \vee q$

用EXISTS/NOT EXISTS实现逻辑蕴涵（2）

［例47］查询至少选修了学生95002选修的全部课程的学生号码

解题思路

- 用逻辑蕴涵表达：查询学号为x的学生，对所有的课程y，只要95002学生选修了课程y，则x也选修了y
- 形式化表示

  用P表示谓词 “学生95002选修了课程y”

  用q表示谓词 “学生x选修了课程y”

  则上述查询：$(\forall y)\ p \rightarrow q$

用EXISTS/NOT EXISTS实现逻辑蕴涵（3）

- 等价变换

  $(\forall y)\ p \rightarrow q \equiv \neg (\exists y\ (\neg (p \rightarrow q)))$

  $\equiv \neg (\exists y\ (\neg (\neg p \vee q))$

  $\equiv \neg \exists y\ (p \wedge \neg q)$

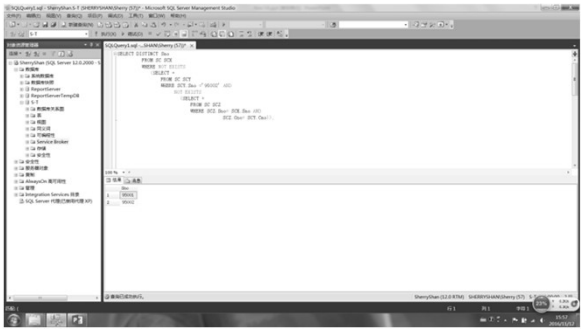
  - 变换后语义：不存在这样的课程y，学生95002选修了y，而学生x没有选

用EXISTS/NOT EXISTS实现逻辑蕴涵（4）

- 用NOT EXISTS谓词表示

```
SELECT DISTINCT Sno
  FROM SC SCX
  WHERE NOT EXISTS
    (SELECT *
      FROM SC SCY
      WHERE SCY.Sno ='95002' AND
        NOT EXISTS
          (SELECT *
            FROM SC SCZ
            WHERE SCZ.Sno= SCX.Sno AND
                  SCZ.Cno= SCY.Cno));
```

运行结果



五、集合查询

- 集合操作的种类
  - 并操作（UNION）
  - 交操作（INTERSECT）
  - 差操作（EXCEPT）

1. 并操作（1）

- 形式

  <查询块>

  UNION

  <查询块>

  - 参加UNION操作的各结果表的列数必须相同，对应项的数据类型也必须相同

1. 并操作（2）

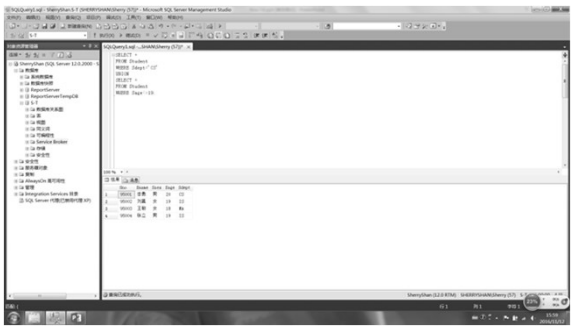
[例48] 查询计算机科学系的学生及年龄不大于19岁的学生

方法一：

```
SELECT *
FROM Student
WHERE Sdept='CS'
UNION
SELECT *
FROM Student
WHERE Sage<=19;
```

- UNION：将多个查询结果合并起来时，系统自动去掉重复元组。
- UNION ALL：将多个查询结果合并起来时，保留重复元组 。

运行结果



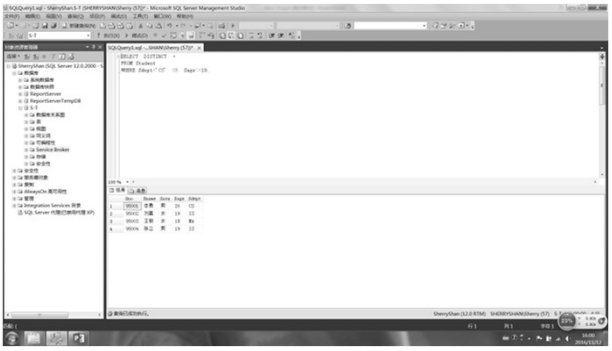
1. 并操作（3）

[例48] 查询计算机科学系的学生及年龄不大于19岁的学生（续）

方法二：

```
SELECT  DISTINCT  *
FROM Student
WHERE Sdept='CS'  OR  Sage<=19;
```

运行结果



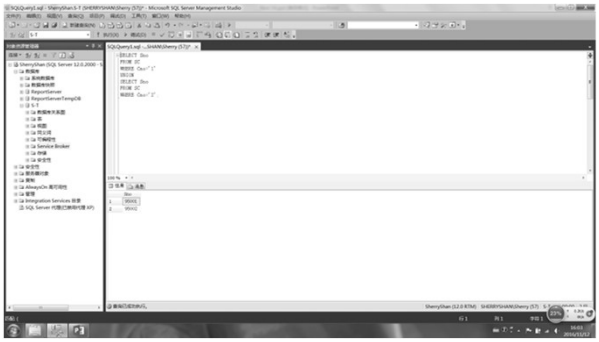
1. 并操作（4）

[例49] 查询选修了课程1或者选修了课程2的学生。

方法一：

```
SELECT Sno
FROM SC
WHERE Cno='1'
UNION
SELECT Sno
FROM SC
WHERE Cno='2';
```

运行结果



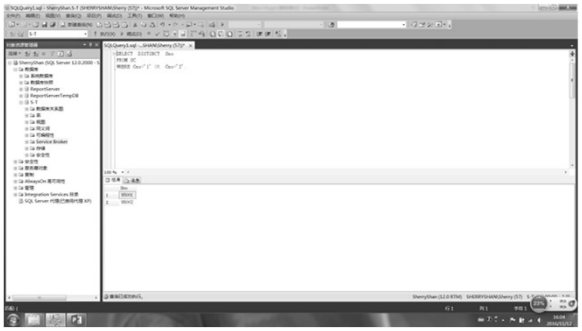
1. 并操作（5）

[例49] 查询选修了课程1或者选修了课程2的学生。（续）

方法二：

```
SELECT  DISTINCT  Sno
FROM SC
WHERE Cno='1' OR  Cno='2';
```

运行结果

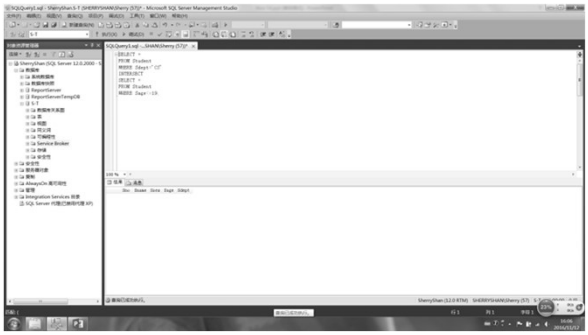


2. 交操作（1）

[例50] 查询计算机科学系的学生与年龄不大于19岁的学生的交集

```
SELECT *
FROM Student
WHERE Sdept='CS'
INTERSECT
SELECT *
FROM Student
WHERE Sage<=19;
```

运行结果

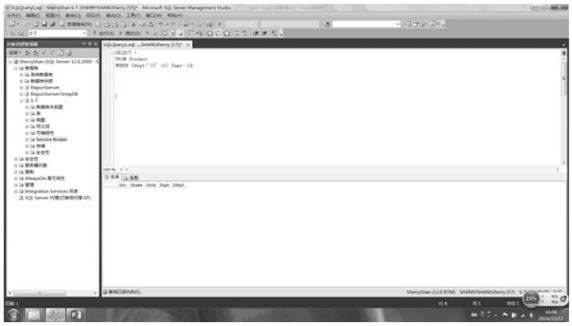


2. 交操作（2）

– 本例实际上是查询计算机科学系中年龄不大于19岁的学生

```
SELECT *
FROM Student
WHERE Sdept='CS' AND Sage<=19;
```

运行结果

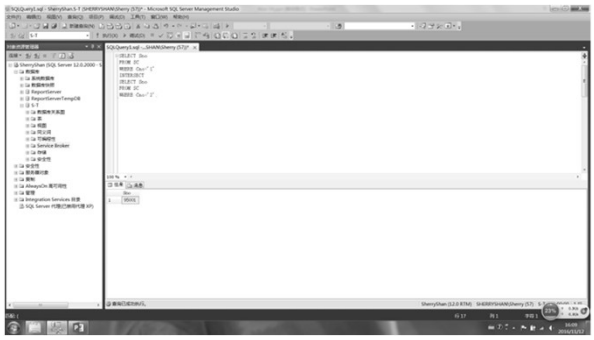


2. 交操作（3）

[例51] 查询既选修了课程1又选修了课程2的学生

```
SELECT Sno
FROM SC
WHERE Cno='1'
INTERSECT
SELECT Sno
FROM SC
WHERE Cno='2';
```

运行结果

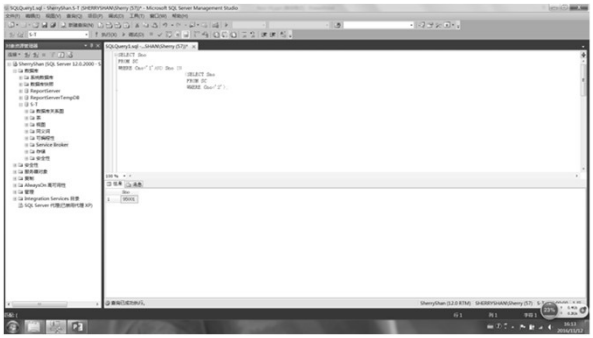


2. 交操作（4）

– 本例也可表示为

```
SELECT Sno
FROM SC
WHERE Cno='1'AND Sno IN
                (SELECT Sno
                 FROM SC
                 WHERE Cno='2');
```

运行结果

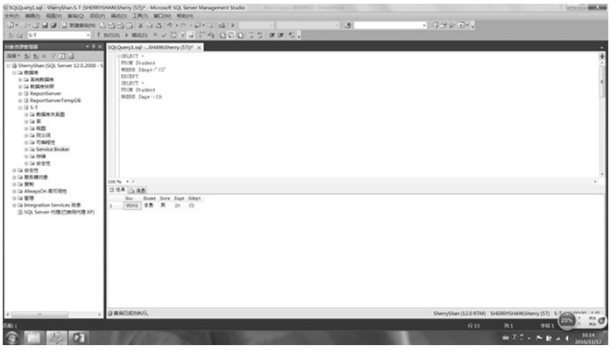


3. 差操作（1）

[例52] 查询计算机科学系的学生与年龄不大于19岁的学生的差集

```
SELECT *
FROM Student
WHERE Sdept='CS'
EXCEPT
SELECT *
FROM Student
WHERE Sage<=19;
```

运行结果



3. 差操作（2）

– 本例实际上是查询计算机科学系中年龄大于19岁的学生

```
SELECT *
FROM Student
WHERE Sdept='CS'AND Sage>19;
```

运行结果

六、SELECT语句的一般格式

```
SELECT [ALL|DISTINCT]
  <目标列表达式> [别名] [ ，<目标列表达式> [别名]] …
FROM    <表名或视图名> [别名]
        [ ，<表名或视图名> [别名]] …
[WHERE <条件表达式>]
[GROUP BY <列名1>[，<列名1'>] ...
[HAVING    <条件表达式>]]
[ORDER BY <列名2> [ASC|DESC]
        [，<列名2'> [ASC|DESC] ] … ]；
```

1. 目标列表达式可选格式

- [<表名>.] *
- [<表名>.]<属性列名表达式>[，[<表名>.]<属性列名表达式>] …
  - <属性列名表达式>：由属性列、作用于属性列的集函数和常量的任意算术运算（+，-，*，/）组成的运算公式

2、聚集函数的一般格式

{COUNT, SUM, AVG, MAX, MIN} （[DISTINCT|ALL] <列名>）

COUNT （[DISTINCT|ALL] *）

3. WHERE子句的条件表达式可选格式（1）

（1）

<属性列名> θ { <属性列名> | <常量> | [ANY|ALL] （SELECT语句） }

3. WHERE子句的条件表达式可选格式（2）

（2）

<属性列名> [NOT] BETWEEN { <属性列名> | <常量> | （SELECT语句） } AND { <属性列名> | <常量> | （SELECT语句） }

3. WHERE子句的条件表达式可选格式（3）

（3）

<属性列名> [NOT] IN { （<值1>[，<值2>] …） | （SELECT语句） }

3. WHERE子句的条件表达式可选格式（4）

（4） <属性列名> [NOT] LIKE <匹配串>

（5）<属性列名> IS [NOT] NULL

（6）[NOT] EXISTS （SELECT语句）

（7）

<条件表达式> { AND | OR } <条件表达式> { AND | OR } <条件表达> …

6.5.2 数据更新

- 一、插入数据
- 二、修改数据
- 三、删除数据
- 四、更新操作与数据库的一致性

一、插入数据

- 两种插入数据方式
  1. 插入单个元组
  2. 插入子查询结果
     - 可以一次插入多个元组

1. 插入单个元组（1）

- 语句格式

  INSERT

  INTO <表名> [（<属性列1>[，<属性列2 >，…]）]

  VALUES （<常量1> [，<常量2>] … ）
- 功能
  – 将新元组插入指定表中

1. 插入单个元组（2）

[例1] 将一个新学生记录（学号：95020；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中

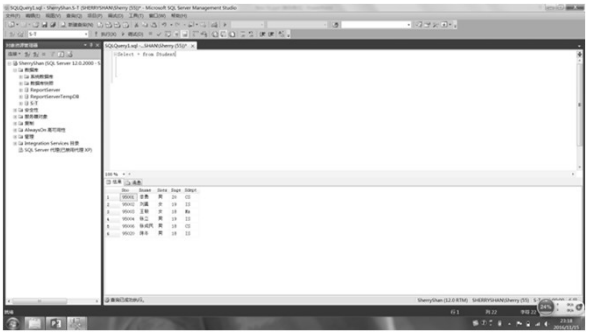
```
INSERT
INTO  Student
VALUES ('95020','陈冬','男',18,'IS');
```

1. 插入单个元组（3）

[例2] 将学生张成民的信息插入到Student表中

```
INSERT
INTO  Student
VALUES ('95006','张成民','男',18,'CS');
```

Student表



1. 插入单个元组（4）

[例3] 插入一条选课记录（‘95020’，‘1 ’）

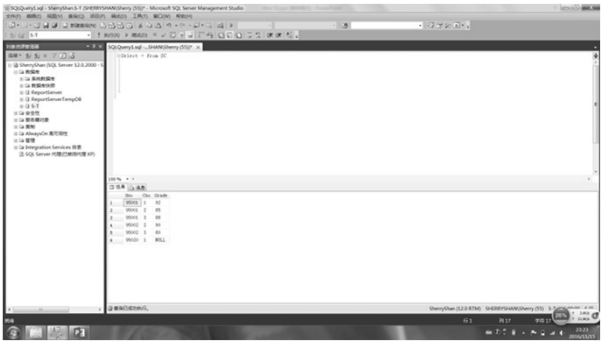
```
INSERT
INTO SC(Sno,Cno)
VALUES ('95020','1');
```

新插入的记录在Grade列上取空值。

或者

```
INSERT
INTO SC
VALUES ('95020','1',NULL);
```

SC表



1. 插入单个元组（5）

- INTO子句
  - 指定要插入数据的表名及属性列
  - 属性列的顺序可与表定义中的顺序不一致
  - 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
  - 指定部分属性列：插入的元组在其余属性列上取空值

1. 插入单个元组（6）

- VALUES子句
  - 提供的值必须与INTO子句匹配
    - 值的个数
    - 值的类型

2. 插入子查询结果（1）

- 语句格式

  INSERT

  INTO <表名> [（<属性列1> [，<属性列2>，…]）]

  子查询；
- 功能
  - 将子查询结果插入指定表中

2. 插入子查询结果（2）

[例4] 对每一个系，求学生的平均年龄，并把结果存入数据库

第一步：建表

```
CREATE  TABLE  Deptage
(Sdept  CHAR(15),        /* 系名*/
Avgage SMALLINT); /*学生平均年龄*/
```

2. 插入子查询结果（3）

第二步：插入数据

```
INSERT
INTO  Deptage(Sdept,Avgage)
SELECT  Sdept,AVG(Sage)
FROM  Student
GROUP BY Sdept;
```

Deptage表

2. 插入子查询结果（4）

- INTO子句（与插入单个元组类似）
  - 指定要插入数据的表名及属性列
  - 属性列的顺序可与表定义中的顺序不一致
  - 没有指定属性列：表示要插入的是一条完整的元组
  - 指定部分属性列：插入的元组在其余属性列上取空值

2. 插入子查询结果（5）

- 子查询
  - SELECT子句目标列必须与INTO子句匹配
    - 值的个数
    - 值的类型

2. 插入子查询结果（6）

- DBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则
  - 实体完整性
  - 参照完整性
  - 用户定义的完整性
    - 对于有NOT NULL约束的属性列是否提供了非空值
    - 对于有UNIQUE约束的属性列是否提供了非重复值
    - 对于有值域约束的属性列所提供的属性值是否在值域范围内

二、修改数据（1）

- 语句格式

  UPDATE <表名>
  SET <列名>=<表达式>[，<列名>=<表达式>]…
  [WHERE <条件>]；
- 功能
  - 修改指定表中满足WHERE子句条件的元组

二、修改数据（2）

- 三种修改方式
  - 修改某一个元组的值
  - 修改多个元组的值
  - 带子查询的修改语句

1. 修改某一个元组的值

[例5] 将学生95001的年龄改为22岁

```
UPDATE  Student
SET Sage=22
WHERE  Sno='95001';
```

2. 修改多个元组的值

[例6] 将所有学生的年龄增加1岁

```
UPDATE Student
SET Sage= Sage+1;
```

3. 带子查询的修改语句

[例7] 将计算机科学系全体学生的成绩置零

```
UPDATE SC
SET  Grade=0
WHERE  'CS'=
        (SELECT Sdept
            FROM  Student
            WHERE  Student.Sno = SC.Sno);
```

二、修改数据（3）

- SET子句
  - 指定修改方式
    - 要修改的列
    - 修改后取值
- WHERE子句
  - 指定要修改的元组
    - 缺省表示要修改表中的所有元组

二、修改数据（4）

- DBMS在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则
  - 实体完整性
    - 主键不允许修改
  - 用户定义的完整性
    - NOT NULL约束
    - UNIQUE约束
    - 值域约束

三、删除数据（1）

- 语句格式

```
DELETE
    FROM    <表名>
    [WHERE <条件>];
```

- 功能
  - 删除指定表中满足WHERE子句条件的元组
- WHERE子句
  - 指定要删除的元组
  - 缺省表示要修改表中的所有元组

三、删除数据（2）

- 三种删除方式
  - 删除某一个元组的值
  - 删除多个元组的值
  - 带子查询的删除语句

三、删除数据（3）

- DBMS在执行删除语句时会检查所删除组是否破坏表上已定义的完整性规则
  - 参照完整性
    - 不允许删除
    - 级联删除

1. 删除某一个元组的值

[例8] 删除学号为95019的学生记录

```
DELETE
FROM Student
WHERE Sno='95019';
```

2. 删除多个元组的值

[例9] 删除所有的学生选课记录

```
DELETE
FROM SC;
```

3. 带子查询的删除语句

[例10] 删除计算机科学系所有学生的选课记录

```
DELETE
FROM SC
WHERE 'CS'=
        (SELECT Sdept
          FROM Student
          WHERE Student.Sno=SC.Sno);
```

四、更新操作与数据库的一致性

- DBMS在执行插入、删除、更新语句时必须保证数据库一致性
  - 必须有事务的概念和原子性
  - 完整性检查和保证

6.6 SQL中的视图（1）

- 视图的特点
  - 虚表，是从一个或几个基本表（或视图）导出的表
  - 只存放视图的定义，不会出现数据冗余
  - 基表中的数据发生变化，从视图中查询出的数据也随之改变

6.6 SQL中的视图（2）

- 基于视图的操作
  - 查询
  - 删除
  - 受限更新
  - 定义基于该视图的新视图

6.6 SQL中的视图（3）

- 6.6.1 定义视图
- 6.6.2 查询视图
- 6.6.3 更新视图
- 6.6.4 视图的作用

6.6.1 定义视图

- 一、建立视图
- 二、删除视图

一、建立视图（1）

- 语句格式

  CREATE VIEW <视图名> [（<列名> [，<列名>]…）]
  AS <子查询>
  [WITH CHECK OPTION]；

一、建立视图（2）

- DBMS执行CREATE VIEW语句时只是把视图的定义存入数据字典，并不执行其中的SELECT语句
- 在对视图查询时，按视图的定义从基本表中将数据查出

一、建立视图（3）

- 组成视图的属性列名
  - 全部省略或全部指定
    - 省略
      - 由子查询中SELECT目标列中的诸字段组成
    - 明确指定视图的所有列名
      - 某个目标列是集函数或列表达式
      - 目标列为 *
      - 多表连接时选出了几个同名列作为视图的字段
      - 需要在视图中为某个列启用新的更合适的名字

一、建立视图（4）

- 常见的视图形式
  1. 行列子集视图
  2. WITH CHECK OPTION的视图
  3. 基于多个基表的视图
  4. 基于视图的视图
  5. 带表达式的视图
  6. 分组视图

1. 行列子集视图

- [例1] 建立信息系学生的视图

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Sage
FROM    Student
WHERE  Sdept='IS';
```

  - 从单个基本表导出
  - 只是去掉了基本表的某些行和某些列
  - 保留了键

2. WITH CHECK OPTION的视图（1）

- 透过视图进行增删改操作时，不得破坏视图定义中的谓词条件（即子查询中的条件表达式）

2. WITH CHECK OPTION的视图（2）

[例2]建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Sage
FROM  Student
WHERE  Sdept='IS'
WITH CHECK OPTION;
```

2. WITH CHECK OPTION的视图（3）

- 对IS_Student视图的更新操作
  - 修改操作
    - DBMS自动加上Sdept= ‘IS’ 的条件
  - 删除操作
    - DBMS自动加上Sdept= ‘IS’ 的条件
  - 插入操作
    - DBMS自动检查Sdept属性值是否为‘IS’
      - 如果不是，则拒绝该插入操作
      - 如果没有提供Sdept属性值，则自动定义Sdept为‘IS’

3. 基于多个基表的视图

[例3] 建立信息系选修了1号课程的学生视图

```
CREATE VIEW IS_S1(Sno,Sname,Grade)
AS
SELECT Student.Sno,Sname,Grade
FROM  Student,SC
WHERE  Sdept= 'IS'AND
          Student.Sno=SC.Sno AND
          SC.Cno= '1';
```

4. 基于视图的视图

[例4] 建立信息系选修了1号课程且成绩在90分以上的学生的视图

```
CREATE VIEW IS_S2
AS
SELECT Sno,Sname,Grade
FROM  IS_S1
WHERE  Grade>=90;
```

5. 带表达式的视图

[例5] 定义一个反映学生出生年份的视图

```
CREATE  VIEW BT_S(Sno,Sname,Sbirth)
AS
SELECT Sno,Sname,1996-Sage  /*假定当年的年份为1996年*/
FROM  Student
```

设置一些派生属性列，也称为虚拟列——Sbirth

- 带表达式的视图必须明确定义组成视图的各个属性列名

6. 分组视图

[例6] 将学生的学号及他的平均成绩定义为一个视图

- 假设SC表中“成绩”列Grade为数字型

```
CREATE  VIEW S_G(Sno,Gavg)
AS
SELECT Sno,AVG(Grade)
FROM  SC
GROUP BY Sno;
```

一、建立视图（5）

- 一类不易扩充的视图
  - 以 SELECT * 方式创建的视图可扩充性差，应尽可能避免

一、建立视图（6）

[例7] 将Student表中所有女生记录定义为一个视图

```
CREATE VIEW
F_Student1(stdnum,name,sex,age,dept)
AS  SELECT *
FROM  Student
WHERE Ssex='女';
```

缺点：修改基表Student的结构后，Student表与F_Student1视图的映象关系被破坏，导致该视图不能正确工作

一、建立视图（7）

```
CREATE VIEW
F_Student2 (stdnum,name,sex,age,dept)
AS  SELECT Sno,Sname,Ssex,Sage,Sdept
FROM  Student
WHERE Ssex='女';
```

为基表Student增加属性列不会破坏Student表与F_Student2视图的映象关系

二、删除视图（1）

- 语句格式

  DROP VIEW <视图名>;
  - 该语句从数据字典中删除指定的视图定义
  - 由该视图导出的其他视图定义仍在数据字典中，但已不能使用，必须显式删除
  - 删除基表时，由该基表导出的所有视图定义都必须显式删除

二、删除视图（2）

[例8]

删除视图BT_S

```
DROP VIEW BT_S;
```

删除视图IS_S1

```
DROP VIEW IS_S1;
```

- 执行此语句时由于IS_S1视图上还导出了IS_S2视图，所以该语句拒绝执行
- 如果确要删除，则使用级联删除语句

  DROP VIEW IS_S1 CASCADE; /*删除了视图IS_S1和由它导出的所有视图*/

6.6.2 查询视图（1）

- 从用户角度：查询视图与查询基本表相同
- DBMS实现视图查询的方法
  - 视图实体化法（View Materialization）
    - 有效性检查：检查所查询的视图是否存在
    - 执行视图定义，将视图临时实体化，生成临时表
    - 查询视图转换为查询临时表
    - 查询完毕删除被实体化的视图（临时表）

6.6.2 查询视图（2）

- DBMS实现视图查询的方法（续）
  - 视图消解法（View Resolution）
    - 进行有效性检查，检查查询的表、视图等是否存在。如果存在，则从数据字典中取出视图的定义
    - 把视图定义中的子查询与用户的查询结合起来，转换成等价的对基本表的查询
    - 执行修正后的查询

6.6.2 查询视图（3）

[例9] 在信息系学生的视图中找出年龄小于20岁的学生

```
SELECT   Sno,Sage
FROM     IS_Student
WHERE    Sage<20;
```

IS_Student视图的定义（定义视图例1）：

6.6.2 查询视图（4）

- 视图实体化法
- 视图消解法

转换后的查询语句为

```
SELECT  Sno,Sage
FROM  Student
WHERE  Sdept='IS'  AND  Sage<20;
```

6.6.2 查询视图（5）

[例10] 查询选修了1号课程的信息系学生

```
SELECT  IS_Student.Sno,Sname
FROM     IS_Student,SC
WHERE  IS_Student.Sno =SC.Sno AND
SC.Cno='1';
```

6.6.2 查询视图（6）

- 视图消解法的局限
  - 有些情况下，视图消解法不能生成正确查询。采用视图消解法的DBMS会限制这类查询

6.6.2 查询视图（7）

[例11]在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *
FROM   S_G
WHERE  Gavg>=90;
```

- S_G视图的子查询定义

```
CREATE VIEW S_G (Sno,Gavg)
AS
SELECT  Sno,AVG(Grade)
FROM  SC
GROUP BY Sno;
```

6.6.2 查询视图（8）

- 错误

```
SELECT Sno,AVG(Grade)
FROM    SC
WHERE  AVG(Grade)>=90
GROUP BY Sno;
```

WHERE子句中不能用聚集函数作为条件表达式

6.6.2 查询视图（9）

- 正确

```
SELECT  Sno,AVG(Grade)
FROM  SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```

6.6.3 更新视图（1）

- 用户角度：更新视图与更新基本表相同
- DBMS实现视图更新的方法
  - 视图实体化法（View Materialization）
  - 视图消解法（View Resolution）
- 指定WITH CHECK OPTION子句
  - DBMS在更新视图时会进行检查，防止用户通过视图对不属于视图范围内的基本表数据进行更新

6.6.3 更新视图（2）

[例12] 将信息系学生视图IS_Student中学号95002的学生姓名改为“刘辰”

```
UPDATE  IS_Student
SET  Sname='刘辰'
WHERE  Sno='95002';
```

转换后的语句

```
UPDATE  Student
SET Sname='刘辰'
WHERE Sno='95002'AND Sdept='IS';
```

6.6.3 更新视图（3）

[例13] 向信息系学生视图IS_S中插入一个新的学生记录：95029，赵新，20岁

```
INSERT
INTO IS_Student
VALUES('95029','赵新',20);
```

转换为对基本表的更新

```
INSERT
INTO  Student(Sno,Sname,Sage,Sdept)
VALUES('95029','赵新',20,'IS');
```

6.6.3 更新视图（4）

[例14]删除信息系学生视图IS_Student中学号为95029的记录

```
DELETE
FROM IS_Student
WHERE Sno='95029';
```

转换为对基本表的更新

```
DELETE
FROM Student
WHERE Sno='95029'AND Sdept='IS';
```

6.6.3 更新视图（5）

- 更新视图的限制
  - 一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新（对两类方法均如此）
    - 例：视图S_G为不可更新视图

```
CREATE VIEW S_G (Sno,Gavg)
AS
SELECT  Sno,AVG(Grade)
FROM    SC
GROUP BY Sno;
```

6.6.3 更新视图（6）

- 对于如下更新语句

```
UPDATE  S_G
SET     Gavg=90
WHERE  Sno='95001';
```

  - 无论实体化法还是消解法都无法将其转换成对基本表SC的更新

6.6.3 更新视图（7）

- 视图的可更新性
  - 理论上可更新
    - 允许更新
    - 不允许更新
  - 理论上不可更新
    - 不允许更新

6.6.3 更新视图（8）

- 实际系统对视图更新的限制
  - 允许对行列子集视图进行更新
  - 对其他类型视图的更新不同系统有不同限制

6.6.4 视图的作用

1. 视图能够简化用户的操作
2. 视图使用户能以多种角度看待同一数据
3. 视图对重构数据库提供了一定程度的逻辑独立性
4. 视图能够对机密数据提供安全保护
5. 适当的利用视图可以更清晰的表达查询

6.7 SQL的数据控制

- 6.7.1 概述
- 6.7.2 授权
- 6.7.3 收回权限
- 6.7.4 小结

6.7.1 概述（1）

- 数据控制亦称为数据保护，包括数据的
  - 安全性控制
  - 完整性控制
  - 并发控制
  - 恢复
- SQL语言提供了数据控制功能，能够在一定程度上保证数据库中数据的安全性、完整性，并提供了一定的并发控制及恢复能力

6.7.1 概述（2）

- 一、完整性
- 二、并发控制
- 三、恢复
- 四、安全性

一、完整性

- 数据库的完整性是指数据库中数据的正确性与相容性
- SQL语言定义完整性约束条件
  - CREATE TABLE语句
  - ALTER TABLE语句
    - 键
    - 取值唯一的列
    - 参照完整性
    - 其他约束条件

二、并发控制

- 并发控制：当多个用户并发地对数据库进行操作时，对他们加以控制、协调，以保证并发操作正确执行，保持数据库的一致性
- SQL语言并发控制能力
  - 提供事务、事务开始、事务结束、提交等概念

三、恢复

- 恢复：当发生各种类型的故障导致数据库处于不一致状态时，将数据库恢复到一致状态的功能
- SQL语言恢复功能
  - 提供事务回滚、重做等概念
    - （UNDO、REDO）

四、安全性（1）

- 安全性：保护数据库，防止不合法的使用所造成的数据泄露和破坏
- 保证数据安全性的主要措施
  - 存取控制：控制用户只能存取他有权存取的数据
  - 规定不同用户对于不同数据对象所允许执行的操作

四、安全性（2）

- DBMS实现数据安全性保护的过程
  - 用户或DBA把授权决定告知系统
  - SQL的GRANT和REVOKE
  - DBMS把授权的结果存入数据字典
  - 当用户提出操作请求时，DBMS根据授权定义进行检查，以决定是否执行操作请求

四、安全性（3）

- 谁定义？
  - DBA和表的建立者（即表的属主）
- 如何定义？
  - SQL语句
    - GRANT
    - REVOKE

6.7.2 授权（1）

- GRANT语句的一般格式

  GRANT <权限>[,<权限>]...
  [ON <对象类型> <对象名>]
  TO <用户>[,<用户>]...
  [WITH GRANT OPTION]；
- 谁定义？
  - DBA和表的建立者（即表的属主）
- 功能：将对指定操作对象的指定操作权限授予指定的用户

6.7.2 授权（2）

- 一、操作权限
- 二、用户的权限
- 三、WITH GRANT OPTION子句
- 四、例题

一、操作权限

对　象	对象类型	操　作　权　限
属性列	TABLE	SELECT, INSERT, UPDATE DELETE, ALLPRIVILEGES
视　图	TABLE	SELECT, INSERT, UPDATE DELETE, ALLPRIVILEGES
基本表	TABLE	SELECT, INSERT, UPDATE DELETE ALTER, INDEX, ALLPRIVILEGES
数据库	DATABASE	CREATETAB

二、用户的权限

- 建表（CREATETAB）的权限：属于DBA
- DBA授予——>普通用户
- 基本表或视图的属主拥有对该表或视图的一切操作权限
- 接受权限的用户
  - 一个或多个具体用户
  - PUBLIC（全体用户）

三、WITH GRANT OPTION子句

- 指定了WITH GRANT OPTION子句
  - 获得某种权限的用户还可以把这种权限再授予别的用户
- 没有指定WITH GRANT OPTION子句
  - 获得某种权限的用户只能使用该权限，不能传播该权限

四、例题（1）

[例1] 把查询Student表权限授给用户U1

```
GRANT   SELECT
ON   TABLE   Student
TO   U1；
```

四、例题（2）

[例2] 把对Student表和Course表的全部权限授予用户U2和U3

```
GRANT ALL PRIVILEGES
ON TABLE Student，Course
TO U2，U3；
```

四、例题（3）

[例3] 把对表SC的查询权限授予所有用户

```
GRANT SELECT
ON TABLE SC
TO  PUBLIC；
```

四、例题（4）

[例4] 把查询Student表和修改学生学号的权限授给用户U4

```
GRANT UPDATE（Sno），SELECT
ON TABLE Student
TO  U4；
```

四、例题（5）

[例5] 把对表SC的INSERT权限授予U5用户，并允许他再将此权限授予其他用户

```
GRANT INSERT
ON TABLE SC
TO  U5
WITH GRANT OPTION；
```

传播权限

- 执行例5后，U5不仅拥有了对表SC的INSERT权限，还可以传播此权限
  GRANT INSERT ON TABLE SC TO U6
  WITH GRANT OPTION；
- 同样，U6还可以将此权限授予U7
  GRANT INSERT ON TABLE SC TO U7；
- 但U7不能再传播此权限
- U5——>U6——>U7

四、例题（6）

[例6] DBA把在数据库S-T中建立表的权限授予用户U8

GRANT CREATETAB
ON DATABASE S-T
TO U8；

6.7.3 收回权限

- REVOKE语句的一般格式为
  REVOKE <权限>[,<权限>]...
  [ON <对象类型> <对象名>]
  FROM <用户>[,<用户>]...；
- 功能：从指定用户那里收回对指定对象的指定权限

例题（1）

[例7] 把用户U4修改学生学号的权限收回

REVOKE UPDATE（Sno）
ON TABLE Student
FROM U4；

例题（2）

[例8] 收回所有用户对表SC的查询权限

REVOKE SELECT
ON TABLE SC
FROM PUBLIC；

例题（3）

[例9] 把用户U5对SC表的INSERT权限收回

REVOKE INSERT
ON TABLE SC
FROM U5；

权限的级联回收

- 系统将收回直接或间接从U5处获得的对SC表的INSERT权限
  ——>U5——> U6——> U7
- 收回U5、U6、U7获得的对SC表的INSERT权限
  <——U5 <—— U6 <——U7

6.7.4 小结

- SQL灵活的授权机制
  - DBA拥有对数据库中所有对象的所有权限，并可以根据应用的需要将不同的权限授予不同的用户
  - 用户对自己建立的基本表和视图拥有全部的操作权限，并且可以用GRANT语句把其中某些权限授予其他用户
  - 被授权的用户如果有“继续授权”的许可，还可以把获得的权限再授予其他用户
  - 所有授予出去的权力在必要时又都可以用REVOKE语句收回

6.8 嵌入式SQL（1）

- SQL语言提供了两种不同的使用方式
  - 交互式
  - 嵌入式
- 为什么要引入嵌入式SQL
  - SQL语言是非过程性语言
  - 事务处理应用需要高级语言
- 这两种方式细节上有差别，在程序设计的环境下，SQL语句要做某些必要的扩充

6.8 嵌入式SQL（2）

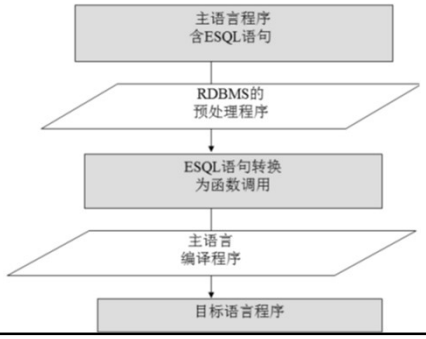
- 6.8.1 嵌入式SQL的处理过程
- 6.8.2 嵌入式SQL与主语言的接口
- 6.8.3 不用游标的嵌入式SQL
- 6.8.4 用游标的嵌入式SQL
- 6.8.5 嵌入式SQL应用实例
- 6.8.6 动态SQL

6.8.1 嵌入式SQL的处理过程（1）

- 主语言
  - 嵌入式SQL是将SQL语句嵌入程序设计语言中，被嵌入的程序设计语言，如C、C++、Java，称为宿主语言，简称主语言
- 处理过程
  - 预编译方法

6.8.1 嵌入式SQL的处理过程（2）

- ESQL基本处理过程



6.8.1 嵌入式SQL的处理过程（3）

- 为了区分SQL语句与主语言语句， 所有SQL语句必须加前缀EXEC SQL，以（；）结束

  EXEC SQL <SQL语句>；

6.8.2 嵌入式SQL与主语言的接口（1）

- 将SQL嵌入到高级语言中混合编程，程序中会含有两种不同计算模型的语句
  - SQL语句
    - 描述性的面向集合的语句
    - 负责操纵数据库
  - 高级语言语句
    - 过程性的面向记录的语句
    - 负责控制程序流程
  - 它们之间应该如何通信？

6.8.2 嵌入式SQL与主语言的接口（2）

- 数据库工作单元与源程序工作单元之间的通信
  - 一、SQL通信区
    - 向主语言传递SQL语句的执行状态信息
    - 使主语言能够据此控制程序流程
  - 二、主变量
    - 主语言向SQL语句提供参数
    - 将SQL语句查询数据库的结果交主语言进一步处理
  - 三、游标
    - 解决集合性操作语言与过程性操作语言的不匹配

一、SQL通信区（1）

- SQLCA： SQL Communication Area
  - SQLCA是一个数据结构
- SQLCA的用途
  - SQL语句执行后，RDBMS反馈给应用程序信息
    - 描述系统当前工作状态
    - 描述运行环境
  - 这些信息将送到SQL通信区SQLCA中
  - 应用程序从SQLCA中取出这些状态信息，据此决定接下来执行的语句

一、SQL通信区（2）

- SQLCA的内容
  - 与所执行的SQL语句有关
  - 与该SQL语句的执行情况有关
    - 例：在执行删除语句DELETE后，不同的执行情况，SQLCA中有不同的信息
      - 违反数据保护规则，操作拒绝
      - 没有满足条件的行，一行也没有删除
      - 成功删除，并有删除的行数
      - 无条件删除警告信息
      - 由于各种原因，执行出错

一、SQL通信区（3）

- SQLCA使用方法
  - 定义SQLCA
    - 用EXEC SQL INCLUDE SQLCA定义
  - 使用SQLCA
    - SQLCA中有一个存放每次执行SQL语句后返回代码的变量SQLCODE
    - 如果SQLCODE等于预定义的常量SUCCESS，则表示SQL语句成功，否则表示出错
    - 应用程序每执行完一条SQL 语句之后都应该测试一下SQLCODE的值，以了解该SQL语句执行情况并做相应处理

二、主变量（1）

- 主变量
  - 嵌入式SQL语句中可以使用主语言的程序变量来输入或输出数据
  - 在SQL语句中使用的主语言程序变量简称为主变量（Host Variable）

二、主变量（2）

- 主变量的类型
  - 输入主变量
    - 由应用程序对其赋值，SQL语句引用
  - 输出主变量
    - 由SQL语句赋值或设置状态信息，返回给应用程序
  - 一个主变量有可能既是输入主变量又是输出主变量

二、主变量（3）

- 主变量的用途
  - 输入主变量
    - 指定向数据库中插入的数据
    - 将数据库中的数据修改为指定值
    - 指定执行的操作
    - 指定WHERE子句或HAVING子句中的条件
  - 输出主变量
    - 获取SQL语句的结果数据
    - 获取SQL语句的执行状态

二、主变量（4）

- 指示变量
  - 一个主变量可以附带一个指示变量（Indicator Variable）
  - 什么是指示变量
    - 整型变量
    - 用来“指示”所指主变量的值或条件
  - 指示变量的用途
    - 输入主变量可以利用指示变量赋空值
    - 输出主变量可以利用指示变量检测出是否空值，值是否被截断

二、主变量（5）

- 在SQL语句中使用主变量和指示变量的方法
  - 1）说明主变量和指示变量
    - BEGIN DECLARE SECTION
    - .....
    - ..... （说明主变量和指示变量）
    - .....
    - END DECLARE SECTION

二、主变量（6）

- 在SQL语句中使用主变量和指示变量的方法（续）
  - 2）使用主变量
    - 说明之后的主变量可以在SQL语句中任何一个能够使用表达式的地方出现
    - 为了与数据库对象名（表名、视图名、列名等）区别，SQL语句中的主变量名前要加冒号（:）作为标志
  - 3）使用指示变量
    - 指示变量前也必须加冒号标志
    - 必须紧跟在所指主变量之后

二、主变量（7）

- 在SQL语句之外（主语言语句中）使用主变量和指示变量的方法
  - 可以直接引用，不必加冒号

三、游标（1）

- 为什么要使用游标
  - SQL语言与主语言具有不同数据处理方式
    - SQL语言是面向集合的，一条SQL语句原则上可以产生或处理多条记录
    - 主语言是面向记录的，一组主变量一次只能存放一条记录
  - 仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求
  - 嵌入式SQL引入了游标的概念，用来协调这两种不同的处理方式

三、游标（2）

- 游标
  - 游标是系统为用户开设的一个数据缓冲区，存放SQL语句的执行结果
  - 每个游标区都有一个名字
  - 用户可以用SQL语句逐一从游标中获取记录，并赋给主变量，交由主语言进一步处理

四、建立和关闭数据库连接

- 嵌入式SQL程序要访问数据库必须先连接数据库。RDBMS根据用户信息对连接请求进行合法性验证，只有通过了身份验证，才能建立一个可用的合法连接
  1. 建立数据库连接
  2. 关闭数据库连接

1. 建立数据库连接（1）

- 建立连接的ESQL语句

  targetEXEC SQL CONNECT TO target [AS connection-name] [USER user-name]；
  - 是要连接的数据库服务器
    - 常见的服务器标识串，如 <dbname>@<hostname>:<port>
    - 包含服务器标识的SQL串常量
    - DEFAULT
  - connect-name是可选的连接名，连接必须是一个有效的标识符
    - 在整个程序内只有一个连接时可以不指定连接名

1. 建立数据库连接（2）

- 如果程序运行过程中建立了多个连接，执行的所有数据库单元的工作都在该操作提交时所选择的当前连接上
- 程序运行过程中可以修改当前连接，对应的嵌入式SQL语句
  - EXEC SQL SET CONNECTION connection-name | DEFAULT；

2. 关闭数据库连接

- 关闭数据库连接

  EXEC SQL DISCONNECT [connection]；

五、程序实例

[例1]依次检查某个系的学生记录，交互式更新某些学生年龄

6.8.3 不用游标的嵌入式SQL

- 不用游标的SQL语句的种类

  一、说明性语句

  二、数据定义语句

  三、数据控制语句

  四、查询结果为单记录的SELECT语句

  五、非CURRENT形式的UPDATE语句

  六、非CURRENT形式的DELETE语句

  七、INSERT语句

一、说明性语句

- 说明性语句是专为在嵌入式SQL中说明主变量、SQLCA等而设置的
- 说明主变量
  1. EXEC SQL BEGIN DECLARE SECTION;
  2. EXEC SQL END DECLARE SECTION;
     - 这两条语句必须配对出现，相当于一个括号，两条语句中间是主变量的说明
- 说明SQLCA
  3. EXEC SQL INCLUDE SQLCA

二、数据定义语句（1）

[例1] 建立一个“学生”表Student

```
EXEC SQL CREATE TABLE Student
        （Sno   CHAR（5）PRIMARY KEY，
         Sname  CHAR（20） UNIQUE，
         Ssex   CHAR（2），
         Sage   SMALLINT，
         Sdept  CHAR（20）
        ）；
```

二、数据定义语句（2）

- 数据定义语句中不允许使用主变量
  - 例：下列语句是错误的

    EXEC SQL DROP TABLE : table_name；

三、数据控制语句

[例2] 把查询Student表权限授给用户U1

```
EXEC SQL GRANT SELECT ON
TABLE Student TO U1；
```

四、查询结果为单记录的SELECT语句（1）

- 语句格式

```
EXEC SQL SELECT  [ALL|DISTINCT]
                 <目标列表达式>[,<目标列表达式>]...
    INTO   <主变量>[<指示变量>]
                 [,<主变量>[<指示变量>]]...
    FROM   <表名或视图名>[,<表名或视图名>] ...
    [WHERE  <条件表达式>]
    [GROUP BY <列名1> [HAVING <条件表达式>]]
    [ORDER BY <列名2> [ASC|DESC]]；
```

四、查询结果为单记录的SELECT语句（2）

- 对交互式SELECT语句的扩充就是多了一个INTO子句
  - 把从数据库中找到的符合条件的记录，放到INTO子句指出的主变量中去

四、查询结果为单记录的SELECT语句（3）

- 使用注意事项
  1. 使用主变量
     - INTO子句
     - WHERE子句的条件表达式
     - HAVING短语的条件表达式

四、查询结果为单记录的SELECT语句（4）

- 使用注意事项（续）
  2. 使用指示变量
     - 指示变量只能用于INTO子句中
     - 如果INTO子句中主变量后面跟有指示变量，则当查询得出的某个数据项为空值时，系统会自动将相应主变量后面的指示变量置为负值，但不向该主变量执行赋值操作，即主变量值仍保持执行SQL语句之前的值
     - 当发现指示变量值为负值时，不管主变量为何值，均应认为主变量值为NULL

四、查询结果为单记录的SELECT语句（5）

- 使用注意事项（续）
  3. 查询结果为空集
     - 如果数据库中没有满足条件的记录，即查询结果为空，则DBMS将SQLCODE的值置为100
  4. 查询结果为多条记录
     - 程序出错，DBMS会在SQLCA中返回错误信息

四、查询结果为单记录的SELECT语句（6）

[例3] 根据学生号码查询学生信息

- 假设已将要查询的学生的学号赋给了主变量givensno

```
EXEC SQL SELECT Sno, Sname, Ssex, Sage, Sdept
        INTO  :Hsno, :Hname, :Hsex, :Hage, :Hdept
        FROM  Student
        WHERE Sno=:givensno；
```

- Hsno, Hname, Hsex, Hage, Hdept和givensno均是主变量，并均已在前面的程序中说明过了。

四、查询结果为单记录的SELECT语句（7）

[例4] 查询某个学生选修某门课程的成绩

- 假设已将要查询的学生的学号赋给了主变量givensno，将课程号赋给了主变量givencno。

```
EXEC SQL SELECT Sno, Cno, Grade
        INTO :Hsno, :Hcno, :Hgrade:Gradeid
                        /*指示变量Gradeid*/
        FROM SC
        WHERE Sno=:givensno AND
                Cno=:givencno；
```

如果Gradeid < 0，不论Hgrade为何值，均认为该学生成绩为空值

四、查询结果为单记录的SELECT语句（8）

- 从提高应用程序的数据独立性角度考虑，SELECT语句在任何情况下都应该使用游标
  - 对于仅返回一行结果数据的SELECT语句虽然可以不使用游标
  - 但如果以后数据库改变了，该SELECT语句可能会返回多行数据，这时该语句就会出错

五、非CURRENT形式的UPDATE语句（1）

- 非CURRENT形式的UPDATE语句
  - 使用主变量
    - SET子句
    - WHERE子句
  - 使用指示变量
    - SET子句
  - 非CURRENT形式的UPDATE语句可以操作多条元组

五、非CURRENT形式的UPDATE语句（2）

[例5] 将全体学生1号课程的考试成绩增加若干分

- 假设增加的分数已赋给主变量Raise

```
EXEC SQL UPDATE SC
        SET Grade=Grade+:Raise
        WHERE Cno='1'；
```

五、非CURRENT形式的UPDATE语句（3）

- [例6] 修改某个学生1号课程的成绩
  - 假设该学生的学号已赋给主变量givensno，修改后的成绩已赋给主变量newgrade

```
EXEC SQL UPDATE SC
        SET Grade=:newgrade
        WHERE Sno=:givensno；
```

五、非CURRENT形式的UPDATE语句（4）

[例7] 将计算机系全体学生年龄置NULL值

```
Sageid=-1；
EXEC SQL UPDATE Student
        SET Sage=:Raise:Sageid
        WHERE Sdept='CS'；
```

将指示变量Sageid赋一个负值后，无论主变量Raise为何值，RDBMS都会将CS系所有学生的年龄置空值

五、非CURRENT形式的UPDATE语句（5）

等价于

```
EXEC SQL UPDATE Student
        SET Sage=NULL
        WHERE Sdept= 'CS' ；
```

六、非CURRENT形式的DELETE语句（1）

- 非CURRENT形式的DELETE语句
  - 使用主变量
    - WHERE子句
- 非CURRENT形式的DELETE语句可以操作多条元组

六、非CURRENT形式的DELETE语句（2）

[例8] 某个学生退学了，现要将有关他的所有选课记录删除掉

- 假设该学生的姓名已赋给主变量stdname

```
EXEC SQL DELETE
        FROM SC
        WHERE Sno=
            （SELECT Sno
             FROM Student
             WHERE Sname=:stdname）；
```

七、INSERT语句（1）

- 非CURRENT形式的INSERT语句
  - 使用主变量
    - VALUES子句
  - 使用指示变量
    - VALUES子句
- 非CURRENT形式的INSERT语句一次只能输入一条元组

七、INSERT语句（2）

- [例9] 某个学生新选修了某门课程，将有关记录插入SC表
  - 假设学生的学号已赋给主变量stdno，课程号已赋给主变量couno

```
gradeid=-1；
EXEC SQL INSERT
    INTO SC（Sno，Cno，Grade）
    VALUES（:stdno，:couno，:gr:gradeid）；
```

    - 由于该学生刚选修课程，尚未考试，因此成绩列为空。所以本例中用指示变量指示相应的主变量为空值

6.8.4 用游标的嵌入式SQL（1）

- 必须使用游标的SQL语句
  - 查询结果为多条记录的SELECT语句
  - CURRENT形式的UPDATE语句
  - CURRENT形式的DELETE语句

6.8.4 用游标的嵌入式SQL（2）

一、 查询结果为多条记录的SELECT语句

二、CURRENT形式的UPDATE和DELETE语句

一、查询结果为多条记录的SELECT语句（1）

- 使用游标的步骤
  1. 定义游标
  2. 打开游标
  3. 推动游标
  4. 关闭游标

1. 定义游标

- 使用DECLARE语句
- 语句格式

  EXEC SQL DECLARE <游标名> CURSOR
  FOR <SELECT语句>；
- 功能
  - 是一条说明性语句，这时DBMS并不执行SELECT指定的查询操作

2. 打开游标

- 使用OPEN语句
- 语句格式

  EXEC SQL OPEN <游标名>；
- 功能
  - 打开游标实际上是执行相应的SELECT语句，把所有满足查询条件的记录从指定表取到缓冲区中
  - 这时游标处于活动状态，指针指向查询结果集中第一条记录之前

3. 推动游标（1）

- 使用FETCH语句
- 语句格式

  EXEC SQL FETCH [[NEXT|PRIOR|
  FIRST|LAST] FROM] <游标名>
  INTO <主变量>[<指示变量>]
  [,<主变量>[<指示变量>]]...；

3. 推动游标（2）

- 功能
  - 指定方向推动游标指针，然后将缓冲区中的当前记录取出来送至主变量供主语言进一步处理
  - NEXT|PRIOR|FIRST|LAST：指定推动游标指针的方式
    - NEXT：向前推进一条记录
    - PRIOR：向回退一条记录
    - FIRST：推向第一条记录
    - LAST：推向最后一条记录
    - 缺省值为NEXT

3. 推动游标（3）

- 说明
  - 主变量必须与SELECT语句中的目标列表达式具有一一对应关系
  - FETCH语句通常用在一个循环结构中，通过循环执行FETCH语句逐条取出结果集中的行进行处理
  - 为进一步方便用户处理数据，现在一些关系数据库管理系统对FETCH语句做了扩充，允许用户向任意方向以任意步长移动游标指针

4. 关闭游标（1）

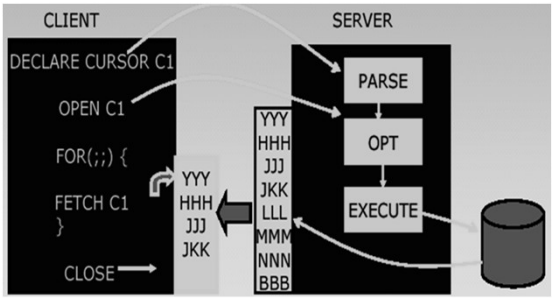
- 使用CLOSE语句
- 语句格式

  EXEC SQL CLOSE <游标名>；
- 功能
  - 关闭游标，释放结果集占用的缓冲区及其他资源

4. 关闭游标（2）

- 说明
  - 游标被关闭后，就不再和原来的查询结果集相联系
  - 被关闭的游标可以再次被打开，与新的查询结果相联系

一、查询结果为多条记录的SELECT语句（2）



例题（1）

[例1] 查询某个系全体学生的信息（学号、姓名、性别和年龄）

```
……
EXEC SQL BEGIN DECLARE SECTION;
……
/* 说明主变量 deptname，HSno，HSname，HSsex，HSage 等*/
……
EXEC SQL END DECLARE SECTION;
……
gets(deptname);                          /* 为主变量 deptname 赋值 */
……
EXEC SQL DECLARE SX CURSOR FOR           /* 说明游标 SX，将 SX
        SELECT Sno，Sname，Ssex，Sage     与查询结果集相联系*/
        FROM    Student
        WHERE   SDept=:deptname;
EXEC SQL OPEN SX                         /* 打开游标 */
WHILE(1)                             /* 用循环结构逐条处理结果集中的记录 */
{
  EXEC SQL FETCH SX INTO :HSno，:HSname，:HSsex，:HSage;
            /* 游标执行向前推进一行，从结果集中取当前行，送相应主变量 */
  if (sqlca.sqlcode <> SUCCESS)          /* 若所有查询结果均已处理完或
    break;                                 出现错误，则退出循环 */
    ……                                 /* 由主语言语句进行进一步处理 */

};
EXEC SQL CLOSE SX;                       /* 关闭游标 SX */
……
```

例题（2）

[例2] 查询某些系全体学生的选课信息

```
……
EXEC SQL BEGIN DECLARE SECTION;
……
        /* 说明主变量 deptname，HSno，HSname，HSsex，HSage 等*/
……
EXEC SQL END DECLARE SECTION;
……
EXEC SQL DECLARE SX CURSOR FOR          /* 说明游标 SX，将 SX
        SELECT Sno，Sname，Ssex，Sage        与查询结果集相联系*/
        FROM Student
        WHERE SDept=:deptname;
WHILE (gets(deptname)!=NULL)            /* 接收主变量 deptname 的值 */
{   /* 处理 deptname 指定系的学生信息，每次循环中 deptname 可具有不同的值 */
    EXEC SQL OPEN SX                    /* 每次打开游标 SX 就和
                                           不同系的学生集合相联系*/
    WHILE (1) {                         /* 用循环结构逐条处理结果集中的记录 */
        EXEC SQL FETCH SX INTO :HSno，:HSname，:HSsex，:HSage;
    /* 游标指针向前推进一行，并后从结果集中取当前行，送相应主变量 */

    if (sqlca.sqlcode <> SUCCESS)       /* 若所有查询结果均已处理完或
        break;                             出现错误，则退出循环 */
    ……                                  /* 在主语言语句进行进一步处理 */
    };                                  /* 内循环结束 */
    EXEC SQL CLOSE SX;                  /* 关闭游标 */
};                                      /* 外循环结束 */
……
```

二、CURRENT形式的UPDATE和DELETE语句（1）

- 用途
  - 非CURRENT形式的UPDATE语句和DELETE语句
    - 面向集合的操作
    - 一次修改或删除所有满足条件的记录
  - 如果只想修改或删除其中某个记录
    - 用带游标的SELECT语句查出所有满足条件的记录
    - 从中进一步找出要修改或删除的记录
    - 用CURRENT形式的UPDATE语句和DELETE语句修改或删除之

二、CURRENT形式的UPDATE和DELETE语句（2）

- 步骤
  - （1） DECLARE
    - 说明游标
  - （2） OPEN
    - 打开游标，把所有满足查询条件的记录从指定表取至缓冲区
  - （3） FETCH
    - 推进游标指针，并把当前记录从缓冲区中取出来送至主变量

二、CURRENT形式的UPDATE和DELETE语句（3）

- 步骤（续）
  - （4） 检查该记录是否是要修改或删除的记录，是则处理之
  - （5） 重复第（3）和（4）步，用逐条取出结果集中的行进行判断和处理
  - （6）CLOSE
    - 关闭游标，释放结果集占用的缓冲区和其他资源

二、CURRENT形式的UPDATE和DELETE语句（4）

- 说明游标
  - 为UPDATE语句说明游标
    - 使用带FOR UPDATE OF <列名>短语的DECLARE语句
    - 语句格式

```
EXEC SQL DECLARE <游标名> CURSOR
        FOR <SELECT语句>
        FOR UPDATE OF <列名>;
```

      - FOR UPDATE OF <列名>短语用于指明检索出的数据在指定列上是可修改的，以便DBMS进行并发控制

二、CURRENT形式的UPDATE和DELETE语句（5）

- 说明游标（续）
  - 为DELETE语句说明游标
    - 使用带FOR UPDATE短语的DECLARE语句
    - 语句格式

```
EXEC SQL DECLARE <游标名> CURSOR
        FOR <SELECT语句>
        FOR UPDATE;
```

      - FOR UPDATE短语提示DBMS进行并发控制

二、CURRENT形式的UPDATE和DELETE语句（6）

- 修改或删除当前记录
  - 经检查缓冲区中记录是要修改或删除的记录，则用UPDATE语句或DELETE语句修改或删除该记录
  - 语句格式

    <UPDATE语句> WHERE CURRENT OF <游标名>

    <DELETE语句> WHERE CURRENT OF <游标名>
    - WHERE CURRENT OF <游标名>子句表示修改或删除的是该游标中最近一次取出的记录

二、CURRENT形式的UPDATE和DELETE语句（7）

- 当游标定义中的SELECT语句带有UNION或ORDER BY子句时，或者该SELECT语句相当于定义了一个不可更新的视图时，不能使用CURRENT形式的UPDATE语句和DELETE语句

例题（1）

[例3] 对某个系的学生信息，根据用户的要求修改其中某些人的年龄字段

- 思路
  - 查询某个系全体学生的信息（要查询的系名由主变量deptname指定）
  - 然后根据用户的要求修改其中某些记录的年龄字段

```
……
EXEC SQL BEGIN DECLARE SECTION;
……
* 说明主变量 deptname，HSno，HSname，HSsex，HSage，NEWAge 等*/
……
EXEC SQL END DECLARE SECTION;
……
gets(deptname);                          * 为主变量 deptname 赋值 */
EXEC SQL DECLARE SX CURSOR FOR

        SELECT Sno，Sname，Ssex，Sage
        FROM Student
        WHERE SDept=:deptname
        FOR UPDATE OF Sage;              /* 说明游标，为 CURRENT UPDATE
                                            作准备 */
EXEC SQL OPEN SX                         /* 打开游标 */
WHILE(1)  {                              /* 用循环结构逐条处理结果集中的记录 */
      EXEC SQL FETCH SX INTO :HSno，:HSname，:HSsex，:HSage;
          /* 游标指针向前推进一行，然后从结果集中取当前行，送相应主变量 */
      if (sqlca.sqlcode <> SUCCESS)
          break;
          /* 若所有查询结果均已处理完或出现 SQL 语句错误，则退出循环 */
      printf("%s，%s，%s，%d", Sno，Sname，Ssex，Sage);   /* 显示该记录 */
      printf("UPDATE AGE ? ");           /* 问用户是否要修改 */
      scanf("%c", &yn);
      if (yn= 'y' or yn= 'Y')            /* 需要修改 */
      {
          printf("INPUT NEW AGE: ");
          scanf("%d", &NEWAge);          /* 输入新的年龄值 */
          EXEC SQL UPDATE Student
                SET Sage = :NEWAge
                WHERE CURRENT OF SX;     /* 修改当前记录的年龄字段 */
      };
      ……
};
EXEC SQL CLOSE SX;                       /* 关闭游标 */
```

例题（2）

[例4] 对某个系的学生信息，根据用户的要求删除其中某些人的记录

```
……
EXEC SQL BEGIN DECLARE SECTION;
    ……
    /* 说明主变量 deptname，HSno，HSname，HSsex，HSage 等*/
    ……
EXEC SQL END DECLARE SECTION;
……
gets(deptname);                          /* 为主变量 deptname 赋值 */
……
EXEC SQL DECLARE SX CURSOR FOR
        SELECT Sno，Sname，Ssex，Sage
        FROM   Student
        WHERE SDept=:deptname;           /* 说明游标 */
EXEC SQL OPEN SX                         /* 打开游标 */
WHILE(1){                                /* 用循环结构逐条处理结果集中的记录 */
    EXEC SQL FETCH SX INTO :HSno，:HSname，:HSsex，:HSage;
        /* 游标指针向前推进一行，然后从结果集中取当前行，送相应主变量 */
    if (sqlca.sqlcode <> SUCCESS)
        break;
        /* 若所有查询结果均已处理完或出现 SQL 语句错误，则退出循环 */
        printf("%s，%s，%s，%d", Sno，Sname，Ssex，Sage);
                                         /* 显示该记录 */
        printf("DELETE ? ");             /* 问用户是否要删除 */
        scanf("%c", &yn);
        if (yn= 'y' or yn= 'Y')          /* 需要删除 */
          EXEC SQL DELETE
                FROM Student
                WHERE CURRENT OF SX;     /* 删除当前记录 */
    ……
};
EXEC SQL CLOSE SX;                       /* 关闭游标 */
……
```

6.8.5 嵌入式SQL应用实例（1）

- 使用游标查询并显示学生选修课的成绩，如果成绩在85分以上，显示成绩为“优”，在75~84之间，成绩为“良”，在60~74之间，成绩为“及格”，在60分以下，成绩为“不及格”

6.8.5 嵌入式SQL应用实例（2）

```
#include <stdio.h>
EXEC SQL BEGIN DECLARE SECTION;          /* 定义主变量 */
CHAR uid(20);
CHAR pwd(20);
CHAR hsno(6);
CHAR hcno(4);
INT hgrade;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE SQLCA;                  /* 定义 SQL 通信区 */
main ()
{char q (6);
 strcpy (uid, "SA");
 strcpy (pwd, "CRC");
 EXEC SQL CONNECT :uid IDENTIFIED BY :pwd        /* 建立与 ISMS 的连接 */
 Scanf ("%s", hcno);                             /* 输入要查询的课程号 */
 EXEC SQL DECLARE C1 CURSOR FOR                  /* (定义游标) */
 SELECT Sno, Grade
 FROM SC
 WHERE Cno= :hcno ;
 EXEC SQL OPEN C1;                               /* (打开游标) */
 while (1)
 {EXEC SQL FETCH C1 INTO :hsno, :hgrade;         /* (推进游标) */
 if (sqlca.sqlcode <>0)
     break;
 if (hgrade>=85)
     q='优';
  else if ((hgrade<85) && ( hgrade>=75))
     q='良';
  else if ((hgrade<75) && ( hgrade>=60))
     q='及格';
  else q='不及格';
 printf("%ns: %s  grade: %s", hsno, q);
};
EXEC SQL CLOSE C1;                               /* (关闭游标) */
EXEC SQL COMMIT WORK RELEASE;                    /* 提交事务,退出 ISMS */
exit(0);
}
```

6.8.6 动态SQL

一、静态嵌入式SQL
二、动态嵌入式SQL

一、静态嵌入式SQL

1. 静态SQL的特点
2. 静态SQL的不足

1. 静态SQL的特点

- 用户可以在程序运行过程中根据实际需要输入WHERE子句或HAVING子句中某些变量的值
- 语句中主变量的个数与数据类型在预编译时都是确定的，只有主变量的值是程序运行过程中动态输入的

2. 静态SQL的不足

- 静态SQL语句提供的编程灵活性在许多情况下仍显得不足，不能编写更为通用的程序
  - 例，对SC
    - 任课教师想查选修某门课程的所有学生的学号及其成绩
    - 班主任想查某个学生选修的所有课程的课程号及相应成绩
    - 学生想查某个学生选修某门课程的成绩
  - 即：查询条件是不确定的，要查询的属性列也是不确定的

二、动态嵌入式SQL（1）

1. 什么是动态嵌入式SQL
   - 动态SQL方法允许在程序运行过程中临时“组装”SQL语句
2. 应用范围
   - 在预编译时下列信息不能确定时
     - SQL语句正文
     - 主变量个数
     - 主变量的数据类型
     - SQL语句中引用的数据库对象（列、索引、基本表、视图等）

二、动态嵌入式SQL（2）

3.动态SQL的形式
   - 语句可变
     - 临时构造完整的SQL语句
   - 条件可变
     - WHERE子句中的条件
     - HAVING短语中的条件
   - 数据库对象、查询条件均可变
     - SELECT子句中的列名
     - FROM子句中的表名或视图名
     - WHERE子句中的条件
     - HAVING短语中的条件

二、动态嵌入式SQL（3）

4. 常用动态SQL语句
   - EXECUTE IMMEDIATE
   - PREPARE
   - EXECUTE
   - DESCRIBE
- 使用动态SQL技术更多的是涉及程序设计方面的知识，而不是SQL语言本身
- 支持动态组装SQL语句和动态参数两种形式

1. 使用SQL语句主变量（1）

- SQL语句主变量
  - 程序主变量包含的内容是SQL语句的内容，而不是原来保存数据的输入或输出变量
  - SQL语句主变量在程序执行期间可以设定不同的SQL语句，然后立即执行

1. 使用SQL语句主变量（2）

[例1] 创建基本表TEST

```
EXEC SQL BEGIN DECLARE SECTION;
const char *stmt = "CREATE TABLE test（a int）；";
            /* SQL语句主变量 */
EXEC SQL END DECLARE SECTION;
... ...
EXEC SQL EXECUTE IMMEDIATE :stmt;
            /* 执行语句 */
```

2. 动态参数（1）

- 动态参数
  - SQL语句中的可变元素
  - 使用参数符号（?）表示该位置的数据在运行时设定
- 和主变量的区别
  - 动态参数的输入不是编译时完成绑定
  - 而是通过（prepare）语句准备主变量和执行（execute）时绑定数据或主变量来完成

2. 动态参数（2）

- 使用动态参数的步骤
  1. 声明SQL语句主变量
  2. 准备SQL语句（PREPARE）

     EXEC SQL PREPARE <语句名> FROM <SQL语句主变量>;
  3. 执行准备好的语句（EXECUTE）

     EXEC SQL EXECUTE <语句名> [INTO <主变量表>] [USING <　主变量或常量>];

2. 动态参数（3）

[例2] 向TEST中插入元组

```
EXEC SQL BEGIN DECLARE SECTION;
const char *stmt = "INSERT INTO test VALUES（?）；";
            /*声明SQL主变量 */
EXEC SQL END DECLARE SECTION;
... ...
EXEC SQL PREPARE mystmt FROM :stmt;   /* 准备语句 */
... ...
EXEC SQL EXECUTE mystmt USING 100;  /* 执行语句 */
EXEC SQL EXECUTE mystmt USING 200;/* 执行语句 */
```

6.9 小结（1）

- SQL的特点
  1. 综合统一
  2. 高度非过程化
  3. 面向集合的操作方式
  4. 同一种语法结构提供两种使用方式
  5. 语言简捷，易学易用

6.9 小结（2）

- 交互式SQL
  - 数据定义
  - 查询
  - 数据更新
  - 数据控制

SQL 语言的动词

SQL 功能	动 词
数据查询	SELECT
数据定义	CREATE, DROP, ALTER
数据操纵	INSERT，UPDATE，DELETE
数据控制	GRANT，REVOKE

6.9 小结（3）

- 嵌入式SQL
  - SQL语句与主语言语句分工非常明确
    - SQL语句
      - 直接与数据库打交道，取出数据库中的数据
    - 主语言语句
      - 控制程序流程
      - 对取出的数据做进一步加工处理

6.9 小结（4）

- 嵌入式SQL（续）
  - 与主语言的通信方式
    1. SQL通信区
       - 向主语言传递SQL语句的执行状态信息
    2. 主变量
       1）主语言向SQL语句提供参数
       2）将SQL语句查询数据库的结果交主语言进一步处理
    3. 游标
       - 解决集合性操作语言与过程性操作语言的不匹配

6.9 小结（5）

- 嵌入式SQL（续）
  - 静态SQL
    - 不用游标
      - 不需要返回结果数据的SQL语句
      - 只返回一条结果的SQL语句
    - 使用游标
      - 定义游标
      - 打开游标
      - 推动游标
      - 关闭游标

6.9 小结（6）

- 嵌入式SQL（续）
  - 动态SQL
    - 动态组装SQL语句
    - 动态参数

参考资料（1）

- 第二、三、四、八章，王珊 萨师煊，数据库系统概论（第5版）， 北京：高等教育出版社，2014
- 第4章，周志逵 郭贵锁等编著，数据库系统原理，北京：清华大学出版社，2008
- 第三章，萨师煊 王珊，数据库系统概论（第三版）， 北京：高等教育出版社，2000

参考资料（2）

- 第1章，[美]Ryan Stephens，Ron Plew，Arie D.Jones著，井中月 郝记生译，SQL入门经典（第5版），北京：人民邮电出版社，2011

Thank You